6. Lab 6 · Décomposer sans rien perdre (1 h 30)

Pourquoi ce lab. Hier, vous avez découpé une table pour supprimer les copies. Mais un découpage peut mal tourner de deux façons : il peut **inventer** des lignes, ou faire **perdre une règle**. Une règle que plus aucune table ne vérifie sera violée un jour, par un bogue ou par un compte volé.

Le cas Queen Bee. L'association reçoit des centres de formation un tableau à plat : quelle apicultrice suit quel formateur, sur quel thème (varroa, récolte, capteurs). Deux règles : un formateur n'enseigne **qu'un** thème ; pour un thème, une apicultrice n'a **qu'un** formateur, celui qui signe son attestation. Si deux formateurs pouvaient signer, l'attestation deviendrait falsifiable.

Dans VS Code, créez `normalisation_j2.md`. Vous y noterez vos réponses tout au long de la journée, une section par étape.

Étape 6.1 · Analyser la table reçue (25 min)

D'où vient cette table ? BeeAfrica forme ses apicultrices sur trois thèmes : le **varroa** (un parasite des abeilles), la **récolte** et les **capteurs**. Les formations sont données par des formateurs extérieurs, dans des centres de formation. Chaque centre envoie à l'association un simple tableau, comme un fichier Excel : une ligne par apicultrice (son téléphone), par thème, avec le nom du formateur. Ce tableau ne vient pas de votre base d'hier : c'est vous qui allez le créer, dans la table `formation_plat`.

Pourquoi l'analyser ? On ne verse jamais un fichier extérieur tel quel dans la base. On cherche d'abord ses règles cachées (les dépendances), ses clés et sa forme normale. Ce tableau cache un piège : il est en 3NF mais pas en BCNF, et si on le découpe, une règle se perd. Vous allez le voir, puis la rétablir.

Exemple GreenFarm · La table et ses dépendances

GreenFarm reçoit le tableau de ses conseillers agricoles. Règles : un conseiller n'a qu'**une** culture ; pour une culture, un exploitant n'a qu'**un** conseiller. On crée la table, puis on vérifie avec `GROUP BY … HAVING` (comme hier au lab 4) que les données ne contredisent pas ces règles.

psql — greenfarm

```
greenfarm=# CREATE TABLE conseil_plat (
greenfarm-#   telephone  text NOT NULL,
greenfarm-#   culture    text NOT NULL,
greenfarm-#   conseiller text NOT NULL,
greenfarm-#   PRIMARY KEY (telephone, culture)
greenfarm-# );
CREATE TABLE
greenfarm=# INSERT INTO conseil_plat VALUES
greenfarm-#   ('06 11 22 33 44', 'tomate', 'Luc Bernard'),
greenfarm-#   ('06 11 22 33 44', 'salade', 'Julie Moreau'),
greenfarm-#   ('06 22 33 44 55', 'tomate', 'Luc Bernard'),
greenfarm-#   ('06 99 88 77 66', 'tomate', 'Sophie Lambert'),
greenfarm-#   ('06 55 44 33 22', 'fraise', 'Marc Dubois'),
greenfarm-#   ('06 33 44 55 66', 'tomate', 'Sophie Lambert');
INSERT 0 6
greenfarm=# SELECT * FROM conseil_plat ORDER BY conseiller, telephone;
   telephone    | culture |   conseiller
----------------+---------+----------------
 06 11 22 33 44 | salade  | Julie Moreau
 06 11 22 33 44 | tomate  | Luc Bernard
 06 22 33 44 55 | tomate  | Luc Bernard
 06 55 44 33 22 | fraise  | Marc Dubois
 06 33 44 55 66 | tomate  | Sophie Lambert
 06 99 88 77 66 | tomate  | Sophie Lambert
(6 rows)
```

La table conseil_plat : 6 lignes

psql — greenfarm

```
greenfarm=# -- (telephone, culture) -> conseiller : la clé
greenfarm-# SELECT telephone, culture FROM conseil_plat
greenfarm-# GROUP BY 1, 2 HAVING count(DISTINCT conseiller) > 1;
 telephone | culture
-----------+---------
(0 rows)
greenfarm=# -- conseiller -> culture
greenfarm-# SELECT conseiller FROM conseil_plat
greenfarm-# GROUP BY conseiller HAVING count(DISTINCT culture) > 1;
 conseiller
------------
(0 rows)
greenfarm=# -- (telephone, conseiller) : une autre clé candidate ?
greenfarm-# SELECT telephone, conseiller FROM conseil_plat
greenfarm-# GROUP BY 1, 2 HAVING count(*) > 1;
 telephone | conseiller
-----------+------------
(0 rows)
```

Trois tests, 0 ligne chacun : aucune règle n'est contredite

Le raisonnement. Deux dépendances : (telephone, culture) → conseiller, et conseiller → culture. Deux clés candidates : (telephone, culture), et (telephone, conseiller), puisque le conseiller donne la culture. Les trois colonnes font partie d'une clé : la table est en **3NF**. Mais conseiller → culture part d'une colonne qui n'est pas une clé : elle n'est **pas en BCNF**.

À vous · La table formation_plat

1. **Dans VS Code**, créez `formation.sql`. En tête : `DROP TABLE IF EXISTS suivi_formation, formateur, formation_plat CASCADE;`, pour pouvoir relancer le fichier. Puis la table `formation_plat` (telephone, theme, formateur, clé primaire (telephone, theme)) et ces huit lignes :

telephone	theme	formateur
+221 77 100 00 01	varroa	Moussa Sarr
+221 77 100 00 01	recolte	Khady Ndiaye
+221 77 100 00 02	varroa	Moussa Sarr
+221 77 200 00 01	varroa	Ibrahima Fall
+221 77 200 00 01	capteurs	Oumar Diallo
+225 07 100 00 01	varroa	Kouassi Yao
+225 07 100 00 02	capteurs	Oumar Diallo
+237 6 77 100 01	recolte	Khady Ndiaye

2. **Dans le terminal** : `psql -U postgres -d queenbee -v ON_ERROR_STOP=1 -f formation.sql`. **Attendu :** `CREATE TABLE`, puis `INSERT 0 8`.
3. **Dans psql**, faites les trois tests de l'exemple avec `theme` et `formateur`. **Attendu :** 0 ligne pour chacun.
4. Dans `normalisation_j2.md` : les deux dépendances, les deux clés candidates, puis « 3NF ? BCNF ? » avec une phrase de justification.

Exemple GreenFarm · L'anomalie

Dans une transaction annulée, on change la culture de Luc Bernard sur **une seule** ligne. La table accepte : Luc Bernard enseigne maintenant deux cultures. La règle conseiller → culture est violée, sans aucun message.

psql — greenfarm

```
greenfarm=# BEGIN;
BEGIN
greenfarm=*# UPDATE conseil_plat SET culture = 'fraise'
greenfarm*# WHERE telephone = '06 22 33 44 55' AND conseiller = 'Luc Bernard';
UPDATE 1
greenfarm=*# SELECT conseiller, array_agg(DISTINCT culture) AS cultures
greenfarm*# FROM conseil_plat GROUP BY conseiller
greenfarm*# HAVING count(DISTINCT culture) > 1;
  conseiller |     cultures
-------------+----------------
 Luc Bernard | {fraise,tomate}
(1 row)
greenfarm=*# ROLLBACK;
ROLLBACK
```

Une ligne modifiée, et la règle est cassée

À vous · Provoquer l'anomalie

Entre `BEGIN;` et `ROLLBACK;`, passez Moussa Sarr en `capteurs` sur la ligne de `+221 77 100 00 02`, puis cherchez les formateurs qui ont plusieurs thèmes. **Attendu :** `Moussa Sarr | {capteurs,varroa}`. Notez-le dans `normalisation_j2.md`.

Étape 6.2 · Découper, et prouver qu'on n'a rien perdu (30 min)

> **La règle pour découper sans perte**
>
> On découpe une table en deux morceaux qui ont une colonne en commun. La jointure des morceaux redonne la table de départ **seulement si** cette colonne commune est la **clé** d'un des deux morceaux. Sinon, la jointure **invente** des lignes. Ces lignes fausses ne se voient pas : c'est le piège.

Exemple GreenFarm · Un mauvais découpage

On découpe en (telephone, culture) et (culture, conseiller). La colonne commune, `culture`, n'est la clé d'aucun morceau : une culture a plusieurs conseillers. La jointure donne 10 lignes au lieu de 6 : 4 lignes **inventées**.

psql — greenfarm

```
greenfarm=# -- une mauvaise décomposition : le lien (culture) n'est la clé d'aucune des deux tables
greenfarm-# CREATE TEMP TABLE m1 AS SELECT DISTINCT telephone, culture FROM conseil_plat;
SELECT 6
greenfarm=# CREATE TEMP TABLE m2 AS SELECT DISTINCT culture, conseiller FROM conseil_plat;
SELECT 4
greenfarm=# SELECT count(*) FROM m1 JOIN m2 USING (culture);
 count
-------
    10
(1 row)
greenfarm=# -- les lignes inventées
greenfarm-# SELECT * FROM m1 JOIN m2 USING (culture)
greenfarm-# EXCEPT
greenfarm-# SELECT culture, telephone, conseiller FROM conseil_plat
greenfarm-# ORDER BY 2;
 culture |   telephone    |   conseiller
---------+----------------+----------------
 tomate  | 06 11 22 33 44 | Sophie Lambert
 tomate  | 06 22 33 44 55 | Sophie Lambert
 tomate  | 06 33 44 55 66 | Luc Bernard
 tomate  | 06 99 88 77 66 | Luc Bernard
(4 rows)
```

10 lignes au lieu de 6 : Paul Martin serait aussi suivi par Sophie Lambert

À vous · Le mauvais découpage

Dans psql, créez deux tables temporaires (`CREATE TEMP TABLE p1 AS SELECT DISTINCT telephone, theme …`, puis `p2` avec theme et formateur), comptez les lignes de leur jointure, puis affichez les lignes inventées avec `EXCEPT`. **Attendu :** 16 lignes au lieu de 8, dont 8 inventées : Fatou Ba (`+221 77 100 00 02`) serait suivie par trois formateurs varroa. Notez pourquoi dans `normalisation_j2.md`. Ne mettez pas ces tables dans `formation.sql` : elles disparaissent en quittant psql.

Exemple GreenFarm · Le bon découpage, et sa preuve

La colonne commune doit être `conseiller`, clé de son morceau. On crée donc deux vraies tables, reliées au reste du schéma par des clés étrangères :

```
CREATE TABLE conseiller (
  id      uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  nom     text NOT NULL UNIQUE,                          -- conseiller -> culture
  culture text NOT NULL CHECK (culture IN ('tomate', 'salade', 'fraise'))
);
CREATE TABLE suivi_conseil (
  exploitant_id uuid NOT NULL REFERENCES exploitant (id),
  conseiller_id uuid NOT NULL REFERENCES conseiller (id),
  PRIMARY KEY (exploitant_id, conseiller_id)
);
```

On les remplit avec `INSERT … SELECT DISTINCT` depuis `conseil_plat`, puis on prouve que rien n'est perdu : on recompose la table et on la compare à l'originale **dans les deux sens**. 0 ligne de différence : rien de perdu, rien d'inventé.

psql — greenfarm

```
greenfarm=# INSERT INTO conseiller (nom, culture)
greenfarm-# SELECT DISTINCT conseiller, culture FROM conseil_plat;
INSERT 0 4
greenfarm=# INSERT INTO suivi_conseil (exploitant_id, conseiller_id)
greenfarm-# SELECT e.id, c.id
greenfarm-# FROM conseil_plat p
greenfarm-# JOIN exploitant e ON e.telephone = p.telephone
greenfarm-# JOIN conseiller c ON c.nom = p.conseiller;
INSERT 0 6
greenfarm=# -- preuve sans perte : 0 ligne attendue
greenfarm-# (SELECT telephone, culture, conseiller FROM conseil_plat
greenfarm-#  EXCEPT
greenfarm-#  SELECT e.telephone, c.culture, c.nom
greenfarm-#  FROM suivi_conseil s
greenfarm-#  JOIN exploitant e ON e.id = s.exploitant_id
greenfarm-#  JOIN conseiller c ON c.id = s.conseiller_id)
greenfarm-# UNION ALL
greenfarm-# (SELECT e.telephone, c.culture, c.nom
greenfarm-#  FROM suivi_conseil s
greenfarm-#  JOIN exploitant e ON e.id = s.exploitant_id
greenfarm-#  JOIN conseiller c ON c.id = s.conseiller_id
greenfarm-#  EXCEPT
greenfarm-#  SELECT telephone, culture, conseiller FROM conseil_plat);
 telephone | culture | conseiller
-----------+---------+------------
(0 rows)
```

4 conseillers, 6 suivis, 0 ligne de différence

À vous · Le découpage BCNF

1. Ajoutez à `formation.sql` une table `formateur` (id, nom **unique**, theme limité à `varroa`, `recolte`, `capteurs`) et une table `suivi_formation` (apicultrice_id vers `apicultrice`, formateur_id vers `formateur`, clé primaire sur les deux).
2. Remplissez-les depuis `formation_plat`, en retrouvant l'apicultrice par son téléphone.

3. Relancez le fichier. **Attendu :** `INSERT 0 5` (cinq formateurs), puis `INSERT 0 8` (huit suivis). Moins de 8 : un téléphone ne correspond à aucune apicultrice, corrigez la ligne.
4. Dans psql, faites la double comparaison `EXCEPT` de l'exemple (recomposez telephone, theme, formateur en joignant `suivi_formation`, `apicultrice` et `formateur`). **Attendu :** 0 ligne.
5. Dans `normalisation_j2.md` : quelle contrainte garantit formateur → theme ? Pourquoi `suivi_formation` pointe-t-elle vers `apicultrice(id)` au lieu de recopier le téléphone ?

Étape 6.3 · La règle perdue, et le trigger qui la rétablit (25 min)

Exemple GreenFarm · La règle perdue

Après le découpage, plus aucune table ne contient à la fois l'exploitant **et** la culture. La règle « un seul conseiller par culture » n'est plus vérifiée : on donne à Paul Martin un second conseiller tomate, et la base accepte.

psql — greenfarm

```
greenfarm=# -- la DF perdue : rien n'empêche un 2e conseiller tomate pour Paul Martin
greenfarm-# INSERT INTO suivi_conseil
greenfarm-# SELECT e.id, c.id FROM exploitant e, conseiller c
greenfarm-# WHERE e.telephone = '06 11 22 33 44' AND c.nom = 'Sophie Lambert';
INSERT 0 1
greenfarm=# SELECT e.nom, c.culture, array_agg(c.nom) AS conseillers
greenfarm-# FROM suivi_conseil s
greenfarm-# JOIN exploitant e ON e.id = s.exploitant_id
greenfarm-# JOIN conseiller c ON c.id = s.conseiller_id
greenfarm-# GROUP BY e.id, e.nom, c.culture
greenfarm-# HAVING count(*) > 1;
     nom     | culture |           conseillers
-------------+---------+---------------------------------
 Paul Martin | tomate  | {"Luc Bernard","Sophie Lambert"}
(1 row)
greenfarm=# DELETE FROM suivi_conseil
greenfarm-# WHERE conseiller_id = (SELECT id FROM conseiller WHERE nom = 'Sophie Lambert')
greenfarm-#   AND exploitant_id = (SELECT id FROM exploitant WHERE telephone = '06 11 22 33 44');
DELETE 1
```

Accepté : Paul Martin a deux conseillers tomate

À vous · Montrer la règle perdue

Inscrivez Fatou Ba chez **Ibrahima Fall** (varroa, alors qu'elle a déjà Moussa Sarr), puis cherchez les apicultrices qui ont deux formateurs pour un même thème (`GROUP BY` apicultrice et thème, `HAVING count(*) > 1`). **Attendu :** `INSERT 0 1`, puis `Fatou Ba | varroa | {"Moussa Sarr","Ibrahima Fall"}`. Supprimez ensuite cette ligne fautive.

> **Le trigger, un gardien programmable**
>
> Un **trigger** est une fonction que PostgreSQL lance **tout seul** quand une ligne est insérée, modifiée ou supprimée. Dans la fonction, `NEW` est la nouvelle ligne. `RETURN NEW;` la laisse passer ; `RAISE EXCEPTION '…';` la refuse et annule toute l'instruction. Le trigger s'applique à **tous** ceux qui écrivent dans la table : l'application, pgAdmin, un script oublié, un attaquant.

Exemple GreenFarm · Le trigger

```
CREATE OR REPLACE FUNCTION suivi_un_conseiller_par_culture() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  IF EXISTS (
    SELECT 1
    FROM suivi_conseil s
    JOIN conseiller c ON c.id = s.conseiller_id
    WHERE s.exploitant_id  = NEW.exploitant_id      -- même exploitant
      AND s.conseiller_id <> NEW.conseiller_id      -- un autre conseiller
      AND c.culture = (SELECT culture FROM conseiller WHERE id = NEW.conseiller_id)
  ) THEN                                            -- … pour la même culture
    RAISE EXCEPTION 'DF violée : cet exploitant a déjà un conseiller pour cette culture';
  END IF;
  RETURN NEW;
END;
$$;

CREATE TRIGGER suivi_conseil_df
BEFORE INSERT OR UPDATE ON suivi_conseil
FOR EACH ROW EXECUTE FUNCTION suivi_un_conseiller_par_culture();
```

```
psql — greenfarm

greenfarm=# -- Paul Martin a déjà Luc Bernard pour la tomate
greenfarm-# INSERT INTO suivi_conseil
greenfarm-# SELECT e.id, c.id FROM exploitant e, conseiller c
greenfarm-# WHERE e.telephone = '06 11 22 33 44' AND c.nom = 'Sophie Lambert';
ERROR:  DF violée : cet exploitant a déjà un conseiller pour cette culture
greenfarm=# -- une culture qu'il n'a pas encore
greenfarm-# INSERT INTO suivi_conseil
greenfarm-# SELECT e.id, c.id FROM exploitant e, conseiller c
greenfarm-# WHERE e.telephone = '06 11 22 33 44' AND c.nom = 'Marc Dubois';
INSERT 0 1
```

Un second conseiller tomate est refusé ; une nouvelle culture est acceptée

À vous · Le trigger de Queen Bee

1. À la fin de `formation.sql`, écrivez la fonction `suivi_un_formateur_par_theme()` et le trigger `suivi_formation_df`, sur le modèle de l'exemple. Relancez le fichier.
2. Testez deux insertions pour Fatou Ba : chez Ibrahima Fall (varroa), puis chez Khady Ndiaye (récolte, thème nouveau pour elle).
3. **Attendu :** la première est refusée (`DF violée : cette apicultrice a déjà un formateur pour ce thème`), la seconde acceptée (`INSERT 0 1`). Copiez les deux messages dans `normalisation_j2.md`.

Pour aller plus loin (facultatif). On peut avoir la même garantie **sans trigger** : on recopie le thème dans `suivi_formation`, avec `UNIQUE (apicultrice_id, theme)` et une clé étrangère composée `(formateur_id, theme) REFERENCES formateur (id, theme)`. Notez dans `normalisation_j2.md` : est-ce encore en BCNF ? Vous verrez au lab 9 pourquoi une contrainte résiste mieux qu'un trigger à deux insertions simultanées.

Étape 6.4 · Relire tout le schéma (10 min)

Exemple GreenFarm · Deux colonnes qui disent peut-être la même chose

`serre` contient `exploitation_id`, et `exploitant` aussi. Est-ce le même fait écrit deux fois ? On regarde les données : chaque exploitant n'a des serres que dans sa propre exploitation.

psql — greenfarm

```
greenfarm=# SELECT e.nom, e.exploitation_id AS exploitation_exploitant,
greenfarm-#        array_agg(DISTINCT s.exploitation_id) AS exploitations_de_ses_serres
greenfarm-# FROM exploitant e JOIN serre s ON s.exploitant_id = e.id
greenfarm-# GROUP BY e.id, e.nom, e.exploitation_id ORDER BY e.nom;
      nom      | exploitation_exploitant | exploitations_de_ses_serres
---------------+-------------------------+-----------------------------
 Claire Petit  | FR-BEAU                 | {FR-BEAU}
 Inès Roux     | FR-LOIR                 | {FR-LOIR}
 Paul Martin   | FR-PROV                 | {FR-PROV}
 Paul Martin   | FR-BEAU                 | {FR-BEAU}
 Pierre Martin | FR-LOIR                 | {FR-LOIR}
(5 rows)
```

Pour chaque exploitant, ses serres sont dans son exploitation

Les données ne suffisent pas à trancher : il faut le **métier**. Si un exploitant ne peut pas avoir de serre ailleurs, `serre.exploitation_id` est une copie qui peut diverger. S'il le peut, ce sont deux faits différents.

À vous · La revue BCNF

1. Dans `normalisation_j2.md`, pour chacune de vos tables : ses dépendances (la clé primaire détermine tout, chaque `UNIQUE` aussi), puis « partie gauche = super-clé ? » et « BCNF ? ».
2. Faites la même requête que l'exemple avec `apicultrice.site_id` et `ruche.site_id`. **Attendu :** chaque apicultrice n'a de ruches que sur son site.
3. Tranchez avec le cahier des charges : règle métier ou coïncidence ? Et dites ce qui garantit la cohérence dans votre schéma.

Enregistrez le lab :

```
git add formation.sql normalisation_j2.md
git commit -m "Lab 6 : décomposition BCNF des formations et trigger"
```

Testez-vous · Lab 6

1. R(A, B, C) avec A → B. Le découpage (A, B) + (A, C) est-il sans perte ? Et (A, B) + (B, C) ?
2. Pourquoi `formation_plat` est-elle en 3NF, alors que formateur → theme ne part pas d'une clé ?
3. Pourquoi le trigger a-t-il besoin de la condition `s.formateur_id <> NEW.formateur_id` ?
4. Un avantage et un inconvénient de rester en 3NF plutôt que de passer en BCNF ?

7. Lab 7 · Dénormaliser, en connaissance de cause (1 h)

Pourquoi ce lab. Hier, vous avez supprimé les copies. Aujourd'hui, vous en créez trois, **exprès** : une synthèse pour aller vite, un export pour un partenaire, un journal pour garder la preuve. Chaque copie a un prix : elle peut être en retard, ou révéler trop de choses.

Copie	Pourquoi	Son prix
table de synthèse (vue matérialisée)	afficher le tableau de bord sans recalculer des millions de mesures	elle est **en retard** jusqu'au prochain rafraîchissement
export pour un tiers	le bailleur veut des chiffres, pas la base	une fuite ne doit révéler **personne**
journal d'audit	garder la preuve, même si la ligne est modifiée ou supprimée	il ne doit **jamais** être modifié

Étape 7.1 · Mesurer avant de copier (15 min)

Exemple GreenFarm · Du volume, puis le tableau de bord

On ajoute 90 jours de relevés horaires pour chaque serre (le fichier ne se lance qu'**une fois**), puis on mesure la requête du tableau de bord avec `EXPLAIN ANALYZE` : il exécute la requête et affiche son temps réel (`Execution Time`).

psql — greenfarm

```
greenfarm=# INSERT INTO releve (serre_id, horodatage, temperature_c, humidite_pct)
greenfarm-# SELECT s.id,
greenfarm-#        TIMESTAMPTZ '2026-12-14 00:00:00+00' + h * INTERVAL '1 hour',
greenfarm-#        round((18 + random() * 10)::numeric, 1),
greenfarm-#        55 + floor(random() * 30)::int
greenfarm-# FROM serre s CROSS JOIN generate_series(0, 90 * 24 - 1) AS h;
INSERT 0 10800
greenfarm=# ANALYZE releve;
ANALYZE
greenfarm=# SELECT count(*) FROM releve;
 count
-------
 10920
(1 row)
```

10 800 relevés ajoutés

```
psql — greenfarm
greenfarm=# SELECT s.exploitation_id, (r.horodatage AT TIME ZONE 'UTC')::date AS jour,
greenfarm-#        count(*) AS nb_releves, round(avg(r.temperature_c), 1) AS temp_moyenne,
greenfarm-#        max(r.temperature_c) AS temp_max
greenfarm-# FROM releve r JOIN serre s ON s.id = r.serre_id
greenfarm-# WHERE s.exploitation_id = 'FR-BEAU'
greenfarm-# GROUP BY 1, 2 ORDER BY jour DESC LIMIT 3;
 exploitation_id |    jour    | nb_releves | temp_moyenne | temp_max
-----------------+------------+------------+--------------+----------
 FR-BEAU         | 2027-03-14 |         48 |         18.5 |     24.6
 FR-BEAU         | 2027-03-13 |         48 |         22.6 |     27.3
 FR-BEAU         | 2027-03-12 |         48 |         22.7 |     27.5
(3 rows)
greenfarm=# EXPLAIN ANALYZE
greenfarm-# SELECT s.exploitation_id, (r.horodatage AT TIME ZONE 'UTC')::date AS jour,
greenfarm-#        count(*) AS nb_releves, round(avg(r.temperature_c), 1) AS temp_moyenne,
greenfarm-#        max(r.temperature_c) AS temp_max
greenfarm-# FROM releve r JOIN serre s ON s.id = r.serre_id
greenfarm-# WHERE s.exploitation_id = 'FR-BEAU'
greenfarm-# GROUP BY 1, 2 ORDER BY jour DESC LIMIT 3;
                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------
-----------------
…
                    ->  Seq Scan on releve r  (cost=0.00..210.20 rows=10920 width=30) (actual time=0.007..0.915 row
s=10920 loops=1)
 Planning Time: 0.158 ms
 Execution Time: 3.783 ms
```

Le tableau de bord lit toute la table des relevés : 3,8 ms

À vous · Le volume et la mesure

1. **Dans VS Code**, créez `volume.sql` : 90 jours de mesures pour chaque ruche, **une toutes les 30 minutes**, à partir du `2026-12-14 00:00:00+00` (`generate_series(0, 90 * 48 - 1)`), avec des valeurs plausibles (poids entre 30 et 35 kg, température intérieure vers 34 °C, batterie à 90 %). Terminez par `ANALYZE mesure;`. Lancez-le **une seule fois**. **Attendu :** `INSERT 0 47520`.
2. Le tableau de bord du gestionnaire (UC-G01) : pour le site `SN-CASA`, par jour, le nombre de mesures, le poids moyen, le poids minimal, la batterie minimale ; les 3 derniers jours. Lancez-la, puis avec `EXPLAIN ANALYZE` devant. **Attendu :** 192 mesures par jour (4 ruches × 48), et un `Execution Time` de l'ordre de 10 ms. Notez-le.

Étape 7.2 · La synthèse, et son prix (15 min)

Exemple GreenFarm · La vue matérialisée

Une vue matérialisée **stocke** son résultat, comme une table : la lecture est immédiate. L'index `UNIQUE` permettra de la rafraîchir sans bloquer les lecteurs.

```
CREATE MATERIALIZED VIEW synthese_exploitation_jour AS
SELECT s.exploitation_id,
       (r.horodatage AT TIME ZONE 'UTC')::date AS jour,
       count(*)                        AS nb_releves,
       round(avg(r.temperature_c), 1) AS temp_moyenne,
       max(r.temperature_c)           AS temp_max
FROM releve r JOIN serre s ON s.id = r.serre_id
GROUP BY s.exploitation_id, (r.horodatage AT TIME ZONE 'UTC')::date;
CREATE UNIQUE INDEX synthese_exploitation_jour_pk
  ON synthese_exploitation_jour (exploitation_id, jour);
```

psql — greenfarm

```
greenfarm=# SELECT * FROM synthese_exploitation_jour
greenfarm-# WHERE exploitation_id = 'FR-BEAU' ORDER BY jour DESC LIMIT 3;
 exploitation_id |    jour    | nb_releves | temp_moyenne | temp_max
-----------------+------------+------------+--------------+----------
 FR-BEAU         | 2027-03-14 |         48 |         18.5 |     24.6
 FR-BEAU         | 2027-03-13 |         48 |         22.6 |     27.3
 FR-BEAU         | 2027-03-12 |         48 |         22.7 |     27.5
(3 rows)
greenfarm=# EXPLAIN ANALYZE
greenfarm-# SELECT * FROM synthese_exploitation_jour
greenfarm-# WHERE exploitation_id = 'FR-BEAU' ORDER BY jour DESC LIMIT 3;
                                                                                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------
---------------
…
         ->  Seq Scan on synthese_exploitation_jour  (cost=0.00..7.41 rows=1 width=108) (actual time=0.003..0.016 r
ows=91 loops=1)
 Planning Time: 0.025 ms
 Execution Time: 0.029 ms
```

Les mêmes chiffres, en 0,03 ms au lieu de 3,8 ms

Le prix : le 15 mars à 6 h, une serre monte à 52 °C. La synthèse ne le montre **pas**, tant qu'on ne l'a pas rafraîchie avec `REFRESH MATERIALIZED VIEW CONCURRENTLY`.

psql — greenfarm

```
greenfarm=# -- 15 mars, 6 h : la serre FR-BEAU-01 monte à 52 °C (chauffage bloqué)
greenfarm-# INSERT INTO releve (serre_id, horodatage, temperature_c, humidite_pct)
greenfarm-# SELECT id, '2027-03-15 06:00+00', 52.0, 20 FROM serre WHERE code = 'FR-BEAU-01';
INSERT 0 1
greenfarm=# SELECT * FROM synthese_exploitation_jour
greenfarm-# WHERE exploitation_id = 'FR-BEAU' ORDER BY jour DESC LIMIT 2;
 exploitation_id |    jour    | nb_releves | temp_moyenne | temp_max
-----------------+------------+------------+--------------+----------
 FR-BEAU         | 2027-03-14 |         48 |         18.5 |     24.6
 FR-BEAU         | 2027-03-13 |         48 |         22.6 |     27.3
(2 rows)
greenfarm=# REFRESH MATERIALIZED VIEW CONCURRENTLY synthese_exploitation_jour;
REFRESH MATERIALIZED VIEW
greenfarm=# SELECT * FROM synthese_exploitation_jour
greenfarm-# WHERE exploitation_id = 'FR-BEAU' ORDER BY jour DESC LIMIT 2;
 exploitation_id |    jour    | nb_releves | temp_moyenne | temp_max
-----------------+------------+------------+--------------+----------
 FR-BEAU         | 2027-03-15 |          1 |         52.0 |     52.0
 FR-BEAU         | 2027-03-14 |         48 |         18.5 |     24.6
(2 rows)
```

Avant le rafraîchissement, rien d'anormal ; après, la journée du 15 mars apparaît

À vous · La synthèse de Queen Bee

1. **Dans VS Code**, créez `denormalisation.sql`, avec la vue matérialisée `synthese_site_jour` (site_id, jour, nb_mesures, poids_moyen, poids_min, batterie_min) et son index unique sur (site_id, jour). Lancez-le.
2. Relisez le tableau de bord dans la vue, avec `EXPLAIN ANALYZE`. **Attendu :** les **mêmes chiffres** qu'à l'étape 7.1, en moins de 0,2 ms. Notez les deux temps.
3. Le 15 mars à 6 h, la ruche `SN-CASA-011` passe à 8,4 kg : 25 kg de moins, c'est un **vol** (UC-CY04). Insérez cette mesure, relisez la vue, rafraîchissez-la, relisez. **Attendu :** le 15 mars n'apparaît qu'après le `REFRESH`, avec `poids_min = 8.40`.
4. Dans `normalisation_j2.md` : si la vue n'est rafraîchie que la nuit, combien de temps le vol reste-t-il invisible ? Où faut-il donc **détecter** un vol : dans la synthèse ou dans les mesures brutes ?

Étape 7.3 · L'export pour le bailleur (20 min)

> **Pseudonymiser, ce n'est pas hacher**
>
> Le bailleur a besoin de chiffres, pas de noms. **Minimiser**, c'est n'exporter que le nécessaire. **Pseudonymiser**, c'est remplacer l'identité par un code. Mais un simple haché (`sha256(telephone)`) se casse en essayant tous les numéros possibles : il n'y en a que quelques millions. Il faut un haché **avec une clé secrète** (HMAC), et la clé ne doit pas être lisible. Une donnée pseudonymisée reste une donnée personnelle au sens du RGPD : celui qui a la clé peut retrouver la personne.

Exemple GreenFarm · Casser un haché simple

On connaît le haché du téléphone de Claire Petit, et le début des numéros (`06 22`). Il reste un million de possibilités : la base les essaie toutes en une seconde.

psql — greenfarm

```
greenfarm=# SELECT nom, encode(sha256(convert_to(telephone, 'UTF8')), 'hex') AS pseudo
greenfarm-# FROM exploitant WHERE nom = 'Claire Petit';
     nom      |                              pseudo
--------------+------------------------------------------------------------------
 Claire Petit | e7e38a00b3cfc490ad43b4884e4756047f44b97f160cea798a73a235e80a8b93
(1 row)
greenfarm=# \timing on
Timing is on.
greenfarm=# SELECT '06 22 ' || substr(to_char(n, 'FM000000'), 1, 2) || ' '
greenfarm-#        || substr(to_char(n, 'FM000000'), 3, 2) || ' '
greenfarm-#        || substr(to_char(n, 'FM000000'), 5, 2) AS telephone_retrouve
greenfarm-# FROM generate_series(0, 999999) AS n
greenfarm-# WHERE encode(sha256(convert_to('06 22 ' || substr(to_char(n, 'FM000000'), 1, 2) || ' '
greenfarm-#        || substr(to_char(n, 'FM000000'), 3, 2) || ' '
greenfarm-#        || substr(to_char(n, 'FM000000'), 5, 2), 'UTF8')), 'hex')
greenfarm-#     = 'e7e38a00b3cfc490…';
 telephone_retrouve
--------------------
 06 22 33 44 55
(1 row)
Time: 1128.933 ms (00:01.129)
```

Le téléphone est retrouvé en 1,1 s

À vous · Casser le pseudonyme de Fatou Ba

Calculez `encode(sha256(convert_to(telephone, 'UTF8')), 'hex')` pour Fatou Ba, copiez le résultat, activez `\timing on`, puis adaptez la requête de l'exemple avec le début `+221 77 1`. **Attendu :** `+221 77 100 00 02` en un peu plus d'une seconde. Notez le temps.

Exemple GreenFarm · Le bon export

La clé est dans une table fermée à tous (`REVOKE ALL … FROM PUBLIC`). La vue calcule un pseudonyme avec `hmac()` et n'expose ni nom, ni téléphone, ni code de serre. On vérifie enfin que la clé n'est pas écrite dans la définition de la vue.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;              -- fournit hmac()
CREATE TABLE cle_pseudonymisation (
  id  int PRIMARY KEY CHECK (id = 1),                 -- une seule ligne
  cle text NOT NULL
);
INSERT INTO cle_pseudonymisation VALUES (1, encode(gen_random_bytes(32), 'hex'));
REVOKE ALL ON cle_pseudonymisation FROM PUBLIC;

CREATE VIEW v_export_recherche AS
SELECT left(encode(hmac(e.id::text, k.cle, 'sha256'), 'hex'), 12) AS exploitant_pseudo,
       x.region, s.surface_m2, count(i.id) AS nb_interventions
FROM serre s
JOIN exploitant e   ON e.id = s.exploitant_id
JOIN exploitation x ON x.id = s.exploitation_id
LEFT JOIN intervention i ON i.serre_id = s.id
CROSS JOIN cle_pseudonymisation k
GROUP BY 1, 2, 3;
```

psql — greenfarm

```
greenfarm=# CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION
greenfarm=# CREATE TABLE cle_pseudonymisation (
greenfarm-#   id  int PRIMARY KEY CHECK (id = 1),
greenfarm-#   cle text NOT NULL
greenfarm-# );
CREATE TABLE
greenfarm=# INSERT INTO cle_pseudonymisation VALUES (1, encode(gen_random_bytes(32), 'hex'));
INSERT 0 1
greenfarm=# REVOKE ALL ON cle_pseudonymisation FROM PUBLIC;
REVOKE
greenfarm=# CREATE VIEW v_export_recherche AS
greenfarm-# SELECT left(encode(hmac(e.id::text, k.cle, 'sha256'), 'hex'), 12) AS exploitant_pseudo,
greenfarm-#        x.region,
greenfarm-#        s.surface_m2,
greenfarm-#        count(i.id) AS nb_interventions
greenfarm-# FROM serre s
greenfarm-# JOIN exploitant e   ON e.id = s.exploitant_id
greenfarm-# JOIN exploitation x ON x.id = s.exploitation_id
greenfarm-# LEFT JOIN intervention i ON i.serre_id = s.id
greenfarm-# CROSS JOIN cle_pseudonymisation k
greenfarm-# GROUP BY 1, 2, 3;
CREATE VIEW
greenfarm=# SELECT * FROM v_export_recherche ORDER BY region;
 exploitant_pseudo |        region       | surface_m2 | nb_interventions
-------------------+---------------------+------------+-----------------
 713428770241      | Centre-Val de Loire |        300 |               1
 136d3f8b708b      | Centre-Val de Loire |        300 |               0
 66c30deb747a      | Centre-Val de Loire |        250 |               0
 47cd5dd02727      | Centre-Val de Loire |        400 |               1
 8b32bf374581      | PACA                |        500 |               1
(5 rows)
greenfarm=# SELECT position((SELECT cle FROM cle_pseudonymisation) IN definition) > 0
greenfarm-#        AS contient_la_cle
greenfarm-# FROM pg_views WHERE viewname = 'v_export_recherche';
 contient_la_cle
-----------------
 f
(1 row)
```

Un pseudonyme, une région, une surface : personne n'est reconnaissable, et la clé n'est pas dans la vue

À vous · L'export du bailleur

1. Dans `denormalisation.sql` : la table `cle_pseudonymisation`, puis la vue `v_export_bailleur` avec le pseudonyme de l'apicultrice (HMAC de son `id`, 12 caractères), le pays, le site, la position **arrondie à 0,1 degré** (`round(r.latitude, 1)`, soit environ 11 km) et le total des récoltes **validées** (`sum(…) FILTER (WHERE rc.statut = 'validee')`).

2. **Attendu :** 8 lignes pour 11 ruches, sans nom ni téléphone, avec des positions comme `12.6 | -16.3`. La vérification de la clé renvoie `f`.
3. Dans `normalisation_j2.md` : quelles colonnes avez-vous retirées, et pourquoi chacune ?

Étape 7.4 · Le journal d'audit, une copie voulue (10 min)

Le journal servira au lab 8. Le modèle est **le même** pour GreenFarm et Queen Bee : recopiez-le à la fin de `denormalisation.sql`.

```
DROP TABLE IF EXISTS journal_audit CASCADE;
CREATE TABLE journal_audit (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  horodatage  timestamptz NOT NULL DEFAULT clock_timestamp(),
  acteur      text NOT NULL,              -- qui
  table_cible text NOT NULL,              -- sur quoi
  operation   text NOT NULL CHECK (operation IN ('INSERT', 'UPDATE', 'DELETE')),
  ligne_id    text,                       -- quelle ligne
  avant       jsonb,                      -- la ligne avant
  apres       jsonb                       -- la ligne après
);
REVOKE ALL ON journal_audit FROM PUBLIC;
```

À vous

Relancez `denormalisation.sql` pour vérifier qu'il est rejouable (ajoutez des `DROP … IF EXISTS` si besoin ; ne relancez **pas** `volume.sql`). Puis répondez dans `normalisation_j2.md` :

1. Pourquoi `ligne_id` n'est-il **pas** une clé étrangère vers la récolte ? (Que deviendrait le journal si la récolte était supprimée ?)
2. `avant` et `apres` recopient des lignes entières : est-ce acceptable ici ?
3. Pourquoi `clock_timestamp()` et pas `now()` ? (Dans une transaction, `now()` donne toujours l'heure du début.)

Enregistrez le lab :

```
git add volume.sql denormalisation.sql normalisation_j2.md
git commit -m "Lab 7 : synthèse, export pseudonymisé, journal d'audit"
```

Testez-vous · Lab 7

1. Quelle différence entre une vue et une vue matérialisée ? Laquelle peut être en retard ?
2. Un collègue propose `md5(nom)` comme pseudonyme. Donnez deux raisons de refuser.
3. Les données de `v_export_bailleur` sont-elles soumises au RGPD ? Pourquoi ?
4. Pourquoi le journal d'audit n'a-t-il pas de clé étrangère vers la table qu'il surveille ?

8. Lab 8 · Contraintes, trigger d'audit et rôles (1 h 30)

Pourquoi ce lab. Une règle écrite seulement dans l'application peut être oubliée, contournée ou bogguée. Ici, c'est la **base** qui fait respecter les règles : ce qui arrive quand on supprime, quelles valeurs sont possibles, qui a fait quoi, et qui a le droit de faire quoi. Puis vous jouez l'attaquant, pour vérifier que tout tient.

Étape 8.1 · Que se passe-t-il quand on supprime ? (15 min)

Action (`ON DELETE`)	Si on supprime la ligne référencée…	Quand l'utiliser
`NO ACTION` (par défaut) ou `RESTRICT`	refus	les lignes liées sont des **preuves** : mesures, récoltes, alertes, inspections
`CASCADE`	les lignes liées sont **supprimées avec**, sans message	lien sans valeur propre : une adhésion
`SET NULL`	la colonne liée devient vide	lien facultatif

Exemple GreenFarm · Le défaut protège, CASCADE efface

Par défaut, les relevés empêchent de supprimer la serre. Avec `CASCADE` (dans une transaction annulée), la même commande efface la serre **et** ses 2 184 relevés, sans un mot.

psql — greenfarm

```
greenfarm=# DELETE FROM serre WHERE code = 'FR-LOIR-02';
ERROR:  update or delete on table "serre" violates foreign key constraint "releve_serre_id_fkey" on table "releve"
DETAIL:  Key (id)=(2b3f99e4-e418-489d-9adf-8675576b0eff) is still referenced from table "releve".
```

Le refus par défaut

psql — greenfarm

```
greenfarm=# BEGIN;
BEGIN
greenfarm=*# ALTER TABLE releve DROP CONSTRAINT releve_serre_id_fkey,
greenfarm*#   ADD CONSTRAINT releve_serre_id_fkey FOREIGN KEY (serre_id)
greenfarm*#   REFERENCES serre (id) ON DELETE CASCADE;
ALTER TABLE
greenfarm=*# SELECT count(*) FROM releve;
 count
-------
 10921
(1 row)
greenfarm=*# DELETE FROM serre WHERE code = 'FR-LOIR-02';
DELETE 1
greenfarm=*# SELECT count(*) FROM releve;
 count
-------
  8737
(1 row)
greenfarm=*# ROLLBACK;
ROLLBACK
```

Avec CASCADE : 10 921 relevés, puis 8 737

On choisit donc : adhésions en `CASCADE`, relevés en `RESTRICT`, puis on vérifie avec une requête sur `pg_constraint`.

psql — greenfarm

```
greenfarm=# ALTER TABLE adhesion
greenfarm-#   DROP CONSTRAINT adhesion_exploitant_id_fkey,
greenfarm-#   ADD  CONSTRAINT adhesion_exploitant_id_fkey FOREIGN KEY (exploitant_id)
greenfarm-#        REFERENCES exploitant (id) ON DELETE CASCADE;
ALTER TABLE
greenfarm=# ALTER TABLE releve
greenfarm-#   DROP CONSTRAINT releve_serre_id_fkey,
greenfarm-#   ADD  CONSTRAINT releve_serre_id_fkey FOREIGN KEY (serre_id)
greenfarm-#        REFERENCES serre (id) ON DELETE RESTRICT;
ALTER TABLE
greenfarm=# SELECT conrelid::regclass AS table_, conname,
greenfarm-#        CASE confdeltype WHEN 'a' THEN 'NO ACTION' WHEN 'r' THEN 'RESTRICT'
greenfarm-#             WHEN 'c' THEN 'CASCADE' WHEN 'n' THEN 'SET NULL' END AS on_delete
greenfarm-# FROM pg_constraint
greenfarm-# WHERE contype = 'f' AND connamespace = 'public'::regnamespace
greenfarm-# ORDER BY on_delete, 1;
     table_     |             conname              | on_delete
----------------+----------------------------------+-----------
 adhesion       | adhesion_exploitant_id_fkey      | CASCADE
 groupement     | groupement_exploitation_id_fkey  | NO ACTION
 exploitant     | exploitant_exploitation_id_fkey  | NO ACTION
 adhesion       | adhesion_groupement_id_fkey      | NO ACTION
 serre          | serre_exploitation_id_fkey       | NO ACTION
 serre          | serre_exploitant_id_fkey         | NO ACTION
 intervention   | intervention_exploitant_id_fkey  | NO ACTION
 intervention   | intervention_serre_id_fkey       | NO ACTION
 compte         | compte_exploitant_id_fkey        | NO ACTION
 suivi_conseil  | suivi_conseil_exploitant_id_fkey | NO ACTION
 suivi_conseil  | suivi_conseil_conseiller_id_fkey | NO ACTION
 releve         | releve_serre_id_fkey             | RESTRICT
(12 rows)
```

Une clé en CASCADE, une en RESTRICT, les autres par défaut

À vous · references.sql

1. Essayez `DELETE FROM ruche WHERE code = 'CI-MAN-012';`. **Attendu :** refus par `mesure_ruche_fk`.
2. Refaites le test `CASCADE` de l'exemple sur `mesure_ruche_fk`, entre `BEGIN;` et `ROLLBACK;`. **Attendu :** `DELETE 1`, et environ 4 368 mesures de moins. Après le `ROLLBACK`, tout revient.
3. **Dans VS Code**, créez `references.sql` : les deux clés de `adhesion` en `CASCADE`, les clés `ruche_id` de `mesure`, `inspection`, `recolte` et `alerte` en `RESTRICT`, chacune avec un commentaire qui justifie le choix. Lancez-le, puis la requête de l'exemple. **Attendu :** 2 `CASCADE`, 4 `RESTRICT`.

Étape 8.2 · Refuser les valeurs impossibles (10 min)

Exemple GreenFarm · Les contraintes CHECK

Le type `smallint` accepte 140 ; une humidité de 140 % est pourtant fausse, ou forgée. `CHECK` la refuse, quel que soit le programme qui écrit.

psql — greenfarm

```
greenfarm=# ALTER TABLE releve
greenfarm-#   ADD CONSTRAINT releve_humidite_0_100 CHECK (humidite_pct BETWEEN 0 AND 100),
greenfarm-#   ADD CONSTRAINT releve_temperature_plausible CHECK (temperature_c BETWEEN -30 AND 70);
ALTER TABLE
greenfarm=# ALTER TABLE compte
greenfarm-#   ADD CONSTRAINT compte_role_valide CHECK (role IN ('exploitant', 'technicien', 'admin'));
ALTER TABLE
greenfarm=# INSERT INTO releve (serre_id, horodatage, temperature_c, humidite_pct)
greenfarm-# SELECT id, '2027-03-15 06:00+00', 21.0, 140 FROM serre WHERE code = 'FR-BEAU-01';
ERROR:  new row for relation "releve" violates check constraint "releve_humidite_0_100"
DETAIL:  Failing row contains (10934, 48956859-b12d-484c-a5f0-4c23e10b2fc3, 2027-03-15 07:00:00+01, 21.0, 140).
greenfarm=# UPDATE compte SET role = 'superadmin' WHERE login = 'c.petit';
ERROR:  new row for relation "compte" violates check constraint "compte_role_valide"
DETAIL:  Failing row contains (f441eba8-868b-4cdc-b5eb-3fb8e54cca4e, c.petit, superadmin, 65655b1e-2ba3-43ec-8f48-b
9e56eb89c91).
```

Une humidité de 140 % et un rôle inconnu sont refusés

À vous · check.sql

Dans VS Code, créez `check.sql` avec ces contraintes, lancez-le, puis testez une mesure avec `batterie_pct = 150` et `UPDATE alerte SET statut = 'fermée' WHERE statut = 'traitee';`.

Table	Contrainte	Règle
`mesure`	`mesure_batterie_0_100`	batterie entre 0 et 100
`mesure`	`mesure_humidite_0_100`	humidité entre 0 et 100
`mesure`	`mesure_poids_positif`	poids positif ou nul
`utilisateur`	`utilisateur_role_valide`	`apicultrice`, `coordinateur`, `gestionnaire` ou `admin`
`alerte`	`alerte_statut_valide`	`ouverte`, `en_cours` ou `traitee`
`recolte`	`recolte_statut_valide`	`declaree`, `validee` ou `ecart`

Attendu : les `ALTER TABLE` passent, les deux tests sont refusés. Si un `ALTER TABLE` est refusé, une ligne existante viole la règle : trouvez-la avec un `SELECT`.

Étape 8.3 · Le trigger d'audit : qui a fait quoi (20 min)

> **Un journal qui sert de preuve**
>
> Le cahier des charges exige que « toutes les actions soient tracées » (UC-AD01). Un journal n'est une preuve que s'il est **écrit par la base** (un trigger, pas l'application), s'il enregistre le **vrai acteur**, et s'il est **en ajout seul** : ni modification, ni suppression.

Exemple GreenFarm · Le code (le même pour Queen Bee)

```
-- 1. Qui agit ? Le rôle choisi par SET ROLE, sinon l'utilisateur connecté
CREATE OR REPLACE FUNCTION qui() RETURNS text
LANGUAGE sql STABLE AS $$
  SELECT CASE WHEN current_setting('role') = 'none' THEN session_user::text
              ELSE current_setting('role') END
$$;

-- 2. Une ligne de journal par ligne touchée
CREATE OR REPLACE FUNCTION audit_ligne() RETURNS trigger
```

```
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_temp AS $$
BEGIN
  INSERT INTO journal_audit (acteur, table_cible, operation, ligne_id, avant, apres)
  VALUES (qui(), TG_TABLE_NAME, TG_OP,
          CASE WHEN TG_OP = 'DELETE' THEN OLD.id::text ELSE NEW.id::text END,
          CASE WHEN TG_OP IN ('UPDATE', 'DELETE') THEN to_jsonb(OLD) END,
          CASE WHEN TG_OP IN ('INSERT', 'UPDATE') THEN to_jsonb(NEW) END);
  RETURN NULL;
END;
$$;
CREATE TRIGGER intervention_audit
AFTER INSERT OR UPDATE OR DELETE ON intervention
FOR EACH ROW EXECUTE FUNCTION audit_ligne();

-- 3. Le journal en ajout seul
CREATE OR REPLACE FUNCTION journal_interdit() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  RAISE EXCEPTION 'journal_audit est en ajout seul : % interdit', TG_OP;
END;
$$;
CREATE TRIGGER journal_ajout_seul
BEFORE UPDATE OR DELETE ON journal_audit
FOR EACH ROW EXECUTE FUNCTION journal_interdit();
CREATE TRIGGER journal_sans_truncate          -- TRUNCATE a besoin de son propre trigger
BEFORE TRUNCATE ON journal_audit
FOR EACH STATEMENT EXECUTE FUNCTION journal_interdit();
```

Trois détails. `SECURITY DEFINER` : la fonction écrit dans le journal avec les droits de son propriétaire ; personne d'autre n'a besoin du droit d'y écrire. `qui()` plutôt que `current_user` : dans une fonction `SECURITY DEFINER`, `current_user` vaut le propriétaire, et le journal accuserait toujours `postgres`. `to_jsonb(OLD)` garde la ligne entière, même si elle est supprimée plus tard.

psql — greenfarm

```
greenfarm=# UPDATE intervention SET commentaire = 'arrosage décalé'
greenfarm-# WHERE serre_id = (SELECT id FROM serre WHERE code = 'FR-BEAU-01');
UPDATE 1
greenfarm=# SELECT acteur, table_cible, operation,
greenfarm-#        avant ->> 'commentaire' AS avant, apres ->> 'commentaire' AS apres
greenfarm-# FROM journal_audit;
  acteur  | table_cible  | operation | avant |      apres
----------+--------------+-----------+-------+-----------------
 postgres | intervention | UPDATE    | RAS   | arrosage décalé
(1 row)
```

Une modification, et sa trace : avant RAS, après « arrosage décalé »

psql — greenfarm

```
greenfarm=# UPDATE journal_audit SET acteur = 'personne';
ERROR:  journal_audit est en ajout seul : UPDATE interdit
greenfarm=# DELETE FROM journal_audit;
ERROR:  journal_audit est en ajout seul : DELETE interdit
greenfarm=# TRUNCATE journal_audit;
ERROR:  journal_audit est en ajout seul : TRUNCATE interdit
greenfarm=# SELECT count(*) FROM journal_audit;
 count
-------
     1
(1 row)
greenfarm=# ALTER TABLE journal_audit DISABLE TRIGGER journal_ajout_seul;
ALTER TABLE
greenfarm=# ALTER TABLE journal_audit ENABLE TRIGGER journal_ajout_seul;
ALTER TABLE
```

Effacer ses traces : trois refus. Mais le propriétaire peut désactiver le trigger

À vous · audit.sql

1. **Dans VS Code**, créez `audit.sql` : les trois fonctions sont **identiques** à l'exemple. Attachez le trigger d'audit à `recolte` (preuve de production, UC-C02) et à `utilisateur` (gestion des comptes), puis protégez le journal. Lancez-le.
2. Modifiez le volume de la récolte de `SN-CASA-011`, puis lisez le journal (`avant ->> 'volume_kg'`, `apres ->> 'volume_kg'`). **Attendu :** `postgres | recolte | UPDATE | 12.5 | 14.0`.
3. Essayez `UPDATE`, `DELETE` et `TRUNCATE` sur le journal. **Attendu :** trois refus, et le nombre de lignes ne bouge pas.
4. En `postgres`, `ALTER TABLE journal_audit DISABLE TRIGGER journal_ajout_seul;` **passe** : réactivez-le aussitôt (`ENABLE TRIGGER`). Notez cette limite dans `matrice_droits.md` : c'est pourquoi le journal doit aussi être copié hors de la base (vers le SIEM du cahier des charges).

Étape 8.4 · Les rôles et les vues de moindre privilège (25 min)

> **Rôles, droits et vues**
>
> Un **rôle** PostgreSQL est un utilisateur (`LOGIN`) ou un groupe (`NOLOGIN`). On donne les droits au **groupe**, puis on met les personnes dans le groupe. Le principe : **tout fermer** (`REVOKE … FROM PUBLIC` ; `PUBLIC` est le groupe dont tout le monde fait partie), puis **ouvrir le minimum**. Une **vue** s'exécute avec les droits de son propriétaire : on peut donc donner à une apicultrice une vue qui ne montre que **ses** lignes, sans aucun droit sur la table. `WITH CHECK OPTION` refuse d'écrire, par la vue, une ligne que la vue ne montrerait pas.

Exemple GreenFarm · roles.sql

```
-- 1. Les groupes, puis les personnes
CREATE ROLE gf_exploitant NOLOGIN;
CREATE ROLE gf_admin      NOLOGIN;
CREATE ROLE "c.petit"         LOGIN PASSWORD 'Serre-cp-2027'    IN ROLE gf_exploitant;
CREATE ROLE "i.roux"          LOGIN PASSWORD 'Serre-ir-2027'    IN ROLE gf_exploitant;
CREATE ROLE "admin.greenfarm" LOGIN PASSWORD 'Serre-admin-2027' IN ROLE gf_admin;

-- 2. Tout fermer, puis ouvrir le minimum
REVOKE CONNECT ON DATABASE greenfarm FROM PUBLIC;
REVOKE ALL ON SCHEMA public FROM PUBLIC;
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;
REVOKE ALL ON ALL FUNCTIONS IN SCHEMA public FROM PUBLIC;
GRANT CONNECT ON DATABASE greenfarm TO gf_exploitant, gf_admin;
GRANT USAGE ON SCHEMA public        TO gf_exploitant, gf_admin;
GRANT EXECUTE ON FUNCTION qui()     TO gf_exploitant, gf_admin;

-- 3. Chacun ne voit que ses lignes, et les colonnes utiles
CREATE VIEW v_moi AS
SELECT id, login, role, exploitant_id FROM compte WHERE login = qui();
CREATE VIEW v_mes_serres AS
SELECT s.id, s.code, s.exploitation_id, s.surface_m2
FROM serre s JOIN compte c ON c.exploitant_id = s.exploitant_id
WHERE c.login = qui();
CREATE VIEW v_mes_interventions AS
SELECT i.id, i.serre_id, i.exploitant_id, i.date_intervention, i.type, i.commentaire
FROM intervention i
WHERE i.serre_id IN (SELECT id FROM v_mes_serres)
  AND i.exploitant_id = (SELECT exploitant_id FROM compte WHERE login = qui())
```

```
WITH CHECK OPTION;

-- 4. Les droits
GRANT SELECT ON v_moi TO gf_exploitant, gf_admin;
GRANT SELECT ON v_mes_serres TO gf_exploitant;
GRANT SELECT, INSERT (serre_id, exploitant_id, date_intervention, type, commentaire)
  ON v_mes_interventions TO gf_exploitant;
GRANT SELECT, INSERT, UPDATE ON compte TO gf_admin;     -- pas de DELETE
GRANT SELECT ON journal_audit TO gf_admin;
```

psql — greenfarm

```
greenfarm=# SELECT g.rolname AS groupe, r.rolname AS personne
greenfarm-# FROM pg_auth_members m
greenfarm-# JOIN pg_roles r ON r.oid = m.member
greenfarm-# JOIN pg_roles g ON g.oid = m.roleid
greenfarm-# WHERE g.rolname LIKE 'gf\_%' ORDER BY 1, 2;
     groupe    |     personne
---------------+----------------
 gf_admin      | admin.greenfarm
 gf_exploitant | c.petit
 gf_exploitant | i.roux
(3 rows)
greenfarm=# \dp v_m*
                                    Access privileges
 Schema |        Name         | Type |     Access privileges     |     Column privileges      | Policies
--------+---------------------+------+---------------------------+----------------------------+----------
 public | v_mes_interventions | view | postgres=arwdDxt/postgres+| serre_id:                 +|
        |                     |      | gf_exploitant=r/postgres  |   gf_exploitant=a/postgres+|
        |                     |      |                           | exploitant_id:            +|
        |                     |      |                           |   gf_exploitant=a/postgres+|
        |                     |      |                           | date_intervention:        +|
        |                     |      |                           |   gf_exploitant=a/postgres+|
        |                     |      |                           | type:                     +|
        |                     |      |                           |   gf_exploitant=a/postgres+|
        |                     |      |                           | commentaire:              +|
        |                     |      |                           |   gf_exploitant=a/postgres |
 public | v_mes_serres        | view | postgres=arwdDxt/postgres+|                            |
        |                     |      | gf_exploitant=r/postgres  |                            |
 public | v_moi               | view | postgres=arwdDxt/postgres+|                            |
        |                     |      | gf_exploitant=r/postgres +|                            |
        |                     |      | gf_admin=r/postgres       |                            |
(3 rows)
```

Les membres des groupes, puis les droits sur les vues (\dp)

Les rôles sont communs à **tout le serveur**. Si `CREATE ROLE` répond `already exists`, le rôle existe déjà : passez à la commande suivante.

À vous · La matrice, puis roles.sql

1. **Dans VS Code**, créez `matrice_droits.md` et remplissez-la **avant** d'écrire le moindre `GRANT`, avec les cas d'utilisation du cahier des charges :

Objet	apicultrice	coordinateur	gestionnaire	admin
ses ruches et leurs mesures	lire	___	___	___
déclarer une récolte	sur ses ruches	___	___	___
valider une récolte (UC-C02)	non	___	___	___
ruches et téléphones de son site	___	___	___	___
synthèse et export (UC-G01, UC-G02)	___	___	___	___
comptes utilisateurs (UC-AD01)	___	___	___	___
journal d'audit	___	___	___	lire

2. **Dans VS Code**, créez `roles.sql`, sur le modèle de l'exemple :
   - quatre groupes : `qb_apicultrice`, `qb_coordinateur`, `qb_gestionnaire`, `qb_admin` ;
   - cinq personnes : `f.ba` et `awa.diop` (apicultrices), `coord.sn` (coordinateur de SN-CASA), `gest.beeafrica` (gestionnaire : ajoutez-le aussi dans `utilisateur`), `admin.beeafrica` ;
   - la fermeture et l'ouverture minimale ;
   - pour l'apicultrice : `v_moi`, `v_mes_ruches`, `v_mes_mesures`, `v_mes_recoltes` (avec `WITH CHECK OPTION`, insertion autorisée sur `ruche_id`, `date_recolte` et `volume_kg` seulement) ;
   - pour le coordinateur : `v_recoltes_site` et `v_ruches_site`, filtrées sur **son** site ;
   - les `GRANT`, conformes à votre matrice.
3. Lancez-le, puis vérifiez les groupes avec la requête de l'exemple (`LIKE 'qb\_%'`). **Attendu :** cinq lignes, une par personne.

Ne retirez **pas** encore le droit `TEMPORARY` sur la base (`REVOKE CONNECT`, pas `REVOKE ALL ON DATABASE`) : le lab 10 en a besoin pour une attaque.

Étape 8.5 · Sans exemple : la validation d'une récolte (10 min)

Le coordinateur doit valider les récoltes **déclarées** de **son** site, et rien d'autre. Lui donner `UPDATE` sur `recolte` serait trop large : il pourrait changer les volumes. Écrivez dans `roles.sql` une fonction `valider_recolte(p_recolte uuid)` qui :

1. est `SECURITY DEFINER`, avec `SET search_path = public, pg_temp` ;
2. retrouve l'utilisateur avec `qui()`, et refuse (`RAISE EXCEPTION`) s'il n'est pas coordinateur ;
3. passe la récolte en `validee`, avec `validee_par` (l'`id` du coordinateur) et `validee_le = now()`, **seulement** si elle est sur une ruche de son site et encore `declaree` ;
4. refuse si aucune ligne n'a changé (`GET DIAGNOSTICS n = ROW_COUNT;`), avec le message `récolte introuvable, déjà traitée ou hors de votre site` ;
5. est fermée à `PUBLIC` et ouverte au seul `qb_coordinateur`.

Étape 8.6 · Essayer de contourner (10 min)

Une protection qui n'a pas été attaquée n'est pas une protection. Deux outils de psql vous servent ici :

Outil	Ce qu'il fait	Exemple
`\gset`	range le résultat d'une requête dans une variable psql	`SELECT id AS ruche_autre FROM ruche WHERE code = 'SN-KOLD-011' \gset`, puis `:'ruche_autre'`
`SET ROLE`	fait agir la session avec les droits d'un autre rôle	`SET ROLE "f.ba";` … `RESET ROLE;` L'invite passe de `=#` à `=>` : vous n'êtes plus superutilisateur

Exemple GreenFarm · Les tests de contournement

En `postgres`, on range d'abord deux identifiants avec `\gset` (`c.petit` n'aura pas le droit de les lire). Puis on devient `c.petit` et on tente tout.

psql — greenfarm

```
greenfarm=# SELECT id AS serre_autre FROM serre WHERE code = 'FR-PROV-01' \gset

greenfarm=# SELECT id AS exploitant_autre FROM exploitant WHERE nom = 'Inès Roux' \gset

greenfarm=# SET ROLE "c.petit";
SET
greenfarm=> SELECT nom, telephone FROM exploitant;
ERROR:  permission denied for table exploitant
greenfarm=> SELECT code FROM v_mes_serres;
    code
------------
 FR-BEAU-02
(1 row)
greenfarm=> INSERT INTO v_mes_interventions (serre_id, exploitant_id, date_intervention, type)
greenfarm-> SELECT id, (SELECT exploitant_id FROM v_moi), '2027-03-16', 'taille' FROM v_mes_serres;
INSERT 0 1
```

Table refusée ; ses serres seulement ; son intervention acceptée

psql — greenfarm

```
greenfarm=> INSERT INTO v_mes_interventions (serre_id, exploitant_id, date_intervention, type)
greenfarm-> VALUES (:'serre_autre', (SELECT exploitant_id FROM v_moi), '2027-03-16', 'taille');
ERROR:  new row violates check option for view "v_mes_interventions"
DETAIL:  Failing row contains (serre_id, exploitant_id, date_intervention, type) = (dcf87bf5-c0d6-490b-b834-fd634c3b
afd7, 65655b1e-2ba3-43ec-8f48-b9e56eb89c91, 2027-03-16, taille).
greenfarm=> INSERT INTO v_mes_interventions (serre_id, exploitant_id, date_intervention, type)
greenfarm-> SELECT id, :'exploitant_autre', '2027-03-16', 'taille' FROM v_mes_serres;
ERROR:  new row violates check option for view "v_mes_interventions"
DETAIL:  Failing row contains (serre_id, exploitant_id, date_intervention, type) = (de132b19-1ec7-4db7-adfa-b8ab6701
8533, 2e5af06e-db76-49c5-b1df-b90f79bd2636, 2027-03-16, taille).
greenfarm=> UPDATE journal_audit SET acteur = 'x';
ERROR:  permission denied for table journal_audit
greenfarm=> RESET ROLE;
RESET
greenfarm=# SELECT acteur, table_cible, operation, apres ->> 'type' AS type
greenfarm-# FROM journal_audit ORDER BY id;
  acteur   | table_cible  | operation |   type
-----------+--------------+-----------+----------
 postgres | intervention | UPDATE    | arrosage
 c.petit  | intervention | INSERT    | taille
(2 rows)
```

Chez un autre ou au nom d'une autre : refusé. Le journal nomme le vrai acteur, c.petit

À vous · tests_contournement.sql

Dans VS Code, créez `tests_contournement.sql`. Pour chaque test : la commande, puis le résultat en commentaire, et « attendu » ou « problème ».

N°	Qui	Tentative	Attendu
1	`f.ba`	lire `apicultrice` directement	refusé
2	`f.ba`	lire `v_mes_ruches`	ses ruches seulement
3	`f.ba`	déclarer une récolte sur sa ruche, par `v_mes_recoltes`	accepté
4	`f.ba`	déclarer une récolte sur la ruche d'une autre (`:'ruche_autre'`)	refusé (`check option`)
5	`f.ba`	déclarer une récolte avec `statut = 'validee'`	refusé
6	`f.ba`	`UPDATE recolte SET volume_kg = 99`	refusé
7	`f.ba`	appeler `valider_recolte(…)`	refusé
8	`f.ba`	écrire une fausse ligne dans `journal_audit`	refusé
9	`f.ba`	`ALTER TABLE recolte DISABLE TRIGGER recolte_audit`	refusé
10	`coord.sn`	valider la récolte du test 3, puis la revalider	accepté, puis refusé

N°	Qui	Tentative	Attendu
11	`coord.sn`	valider une récolte d'un autre site (CM-BAF)	refusé
12	`admin.beeafrica`	lire le journal ; supprimer un utilisateur	lu ; refusé

Attendu : tous les refus ont lieu (`permission denied…`, `new row violates check option…`, `must be owner of table recolte`, et votre message de l'étape 8.5). Lu par l'admin, le journal contient `f.ba | recolte | INSERT` et `coord.sn | recolte | UPDATE` : les **vrais acteurs**, même quand la validation passe par une fonction exécutée avec les droits de `postgres`.

Enregistrez le lab :

```
git add references.sql check.sql audit.sql roles.sql tests_contournement.sql matrice_droits.md
git commit -m "Lab 8 : contraintes, audit, rôles et tests de contournement"
```

Testez-vous · Lab 8

1. Pourquoi `ON DELETE CASCADE` sur `mesure.ruche_id` est-il un risque pour la preuve ?
2. Que vaut `current_user` dans une fonction `SECURITY DEFINER` qui appartient à `postgres` ? Quelle conséquence pour le journal ?
3. Pourquoi donner à l'apicultrice `INSERT` sur la vue `v_mes_recoltes` plutôt que sur la table `recolte` ?
4. Que risque-t-on si l'on oublie `REVOKE ALL ON FUNCTION valider_recolte(uuid) FROM PUBLIC` ?

9. Lab 9 · Transactions : tout ou rien, et l'un après l'autre (1 h 30)

Pourquoi ce lab. Deux dangers. Une action peut réussir à moitié : la trace dit « fait », la base dit « pas fait ». Et deux requêtes peuvent arriver **en même temps** : un jeton prévu pour servir une fois sert alors deux fois. Vous allez provoquer ces deux bogues, puis les corriger.

> **Les propriétés ACID**
>
> Une **transaction** regroupe des instructions : `BEGIN;` … `COMMIT;` (valider) ou `ROLLBACK;` (annuler). Sans `BEGIN`, chaque instruction est validée aussitôt (autocommit). **A**tomicité : tout ou rien. **C**ohérence : les contraintes sont toujours respectées. **I**solation : deux transactions simultanées ne se gênent pas… selon le niveau choisi. **D**urabilité : après `COMMIT`, la donnée survit à une panne.

Étape 9.1 · L'action et sa trace ensemble (15 min)

Exemple GreenFarm · La trace qui ment, puis la transaction

Une application qui écrit son propre journal **avant** d'agir : le journal dit « passé superviseur », mais la base a refusé le changement. La trace ment.

psql — greenfarm

```
greenfarm=# CREATE TEMP TABLE journal_appli (quand timestamptz DEFAULT now(), message text);
CREATE TABLE
greenfarm=# INSERT INTO journal_appli (message) VALUES ('compte c.petit passé superviseur');
INSERT 0 1
greenfarm=# UPDATE compte SET role = 'superviseur' WHERE login = 'c.petit';
ERROR:  new row for relation "compte" violates check constraint "compte_role_valide"
DETAIL:  Failing row contains (f441eba8-868b-4cdc-b5eb-3fb8e54cca4e, c.petit, superviseur, 65655b1e-2ba3-43ec-8f48-
b9e56eb89c91).
greenfarm=# SELECT message FROM journal_appli;
             message
---------------------------------
 compte c.petit passé superviseur
(1 row)
greenfarm=# SELECT login, role FROM compte WHERE login = 'c.petit';
  login  |    role
---------+------------
 c.petit | exploitant
(1 row)
```

Le journal de l'application affirme un changement qui n'a pas eu lieu

Avec une transaction et le trigger d'audit, l'action et sa trace partent **ensemble**. Après une erreur, l'invite devient `=!#` et toute commande est refusée jusqu'au `ROLLBACK` ; puis tout revient à l'état initial.

psql — greenfarm

```
greenfarm=# SELECT count(*) FROM journal_audit;
 count
-------
     2
(1 row)
greenfarm=# BEGIN;
BEGIN
greenfarm=*# UPDATE compte SET role = 'technicien' WHERE login = 'i.roux';
UPDATE 1
greenfarm=*# SELECT count(*) FROM journal_audit;
 count
-------
     3
(1 row)
greenfarm=*# SELECT 1 / 0 AS panne_simulee;
ERROR:  division by zero
greenfarm=!# SELECT count(*) FROM journal_audit;
ERROR:  current transaction is aborted, commands ignored until end of transaction block
greenfarm=!# ROLLBACK;
ROLLBACK
greenfarm=# SELECT count(*) FROM journal_audit;
 count
-------
     2
(1 row)
greenfarm=# SELECT role FROM compte WHERE login = 'i.roux';
    role
------------
 exploitant
(1 row)
```

Le journal passe de 2 à 3, puis revient à 2 avec le ROLLBACK

À vous

1. Créez une table temporaire `journal_appli`, écrivez-y `récolte SN-CASA-011 validée par coord.sn`, puis `UPDATE recolte SET statut = 'validee'` sur cette ruche, sans `validee_par`. **Attendu :** l'`UPDATE` est refusé par `recolte_validation_tracee`, mais le journal de l'application affirme la validation.
2. Comptez les lignes de `journal_audit`. Puis, entre `BEGIN;` et `ROLLBACK;` : la validation complète (avec `validee_par` et `validee_le`), un nouveau comptage, `SELECT 1 / 0;`, un comptage. **Attendu :** +1 dans la transaction, `current transaction is aborted` après l'erreur, et tout revenu après le `ROLLBACK`.
3. **Dans VS Code**, créez `concurrence.md` et notez ces deux résultats.

Étape 9.2 · La course au jeton, dans deux terminaux (25 min)

> **« Vérifier, puis agir »**
>
> Le cahier des charges prévoit des **jetons de rafraîchissement** (UC-T01), qui ne doivent servir **qu'une fois**. Le code naïf fait deux étapes : il **vérifie** que le jeton n'a pas servi, puis il **agit** (il le marque utilisé et ouvre une session). Si un attaquant qui a volé le jeton l'envoie **en même temps** que l'application, les deux requêtes vérifient avant que l'une n'agisse : deux sessions s'ouvrent. On appelle cela une **race condition**.

Exemple GreenFarm · Le lien de réinitialisation utilisé deux fois

GreenFarm envoie un lien de réinitialisation de mot de passe, valable une fois. On crée la table et 20 liens de test :

```
CREATE TABLE jeton_reinit (
  id         uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  compte_id  uuid NOT NULL REFERENCES compte (id),
  valeur     text NOT NULL UNIQUE,
  expire_le  timestamptz NOT NULL,
  utilise_le timestamptz                    -- vide tant que le lien n'a pas servi
);
CREATE TABLE changement_mdp (
  id        uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  compte_id uuid NOT NULL REFERENCES compte (id),
  jeton_id  uuid NOT NULL REFERENCES jeton_reinit (id),
  change_le timestamptz NOT NULL DEFAULT clock_timestamp()
);
INSERT INTO jeton_reinit (compte_id, valeur, expire_le)
SELECT id, 'RESET-' || login || '-' || n, now() + INTERVAL '15 minutes'
FROM compte CROSS JOIN generate_series(1, 20) AS n
WHERE login = 'c.petit';
```

Puis deux terminaux, `psql -U postgres -d greenfarm` dans chacun, et les commandes dans l'ordre des temps T1, T2… Les deux `SELECT` voient le lien libre ; les deux changements passent.

Deux terminaux psql — vérifier, puis agir

	Terminal A (clic 1)	Terminal B (clic 2)
T1	`greenfarm=# BEGIN;` `BEGIN`	
T2		`greenfarm=# BEGIN;` `BEGIN`

T3	`greenfarm=*# SELECT valeur, utilise_le FROM jeton_reinit`
	`greenfarm*# WHERE valeur = 'RESET-c.petit-1';`
	`valeur | utilise_le`
	`----------+----------`
	`RESET-c.petit-1 |`
	`(1 row)`	

T4		`greenfarm=*# SELECT valeur, utilise_le FROM jeton_reinit`
		`greenfarm*# WHERE valeur = 'RESET-c.petit-1';`
		`valeur | utilise_le`
		`----------+----------`
		`RESET-c.petit-1 |`
		`(1 row)`

T5	`greenfarm=*# UPDATE jeton_reinit SET utilise_le = now()`
	`greenfarm*# WHERE valeur = 'RESET-c.petit-1';`
	`UPDATE 1`	

T6	`greenfarm=*# INSERT INTO changement_mdp (compte_id, jeton_id)`
	`greenfarm*# SELECT compte_id, id FROM jeton_reinit WHERE valeur = 'RESET-c.petit-1';`
	`INSERT 0 1`	

T7	`greenfarm=*# COMMIT;` `COMMIT`	

T8		`greenfarm=*# UPDATE jeton_reinit SET utilise_le = now()`
		`greenfarm*# WHERE valeur = 'RESET-c.petit-1';`
		`UPDATE 1`

T9		`greenfarm=*# INSERT INTO changement_mdp (compte_id, jeton_id)`
		`greenfarm*# SELECT compte_id, id FROM jeton_reinit WHERE valeur = 'RESET-c.petit-1';`
		`INSERT 0 1`

T10		`greenfarm=*# COMMIT;` `COMMIT`

T11	`greenfarm=# SELECT count(*) AS changements FROM changement_mdp c`
	`greenfarm-# JOIN jeton_reinit j ON j.id = c.jeton_id WHERE j.valeur = 'RESET-c.petit-1';`
	`changements`
	`------------`
	`2`
	`(1 row)`	

Deux changements de mot de passe avec un seul lien

À vous · jeton.sql et la course

1. **Dans VS Code**, créez `jeton.sql` : une table `jeton` (id, utilisateur_id vers `utilisateur`, valeur unique, expire_le, utilise_le), une table `session_app` (id, utilisateur_id, jeton_id vers `jeton`, ouverte_le par défaut

`clock_timestamp()`), et trente jetons pour `f.ba`, de `RT-f.ba-1` à `RT-f.ba-30`, valables un jour. **Attendu :** `INSERT 0 30`.

2. Ouvrez **deux** terminaux et jouez la course avec `RT-f.ba-1`, comme dans l'exemple : `BEGIN`, `SELECT` du jeton, `UPDATE`, `INSERT INTO session_app … SELECT utilisateur_id, id FROM jeton WHERE valeur = 'RT-f.ba-1'`, `COMMIT`. **Attendu :** 2 sessions avec ce jeton.
3. Recopiez dans `concurrence.md` le tableau des temps, avec ce que chaque terminal a répondu.

Étape 9.3 · La course par un programme (15 min)

Exemple GreenFarm · Deux clics partis ensemble

`course_reinit.py` lance deux fils d'exécution qui attendent à une barrière, puis partent ensemble. Chacun vérifie, attend un peu (l'envoi d'un e-mail, par exemple), puis agit :

```
"""course_reinit.py (GreenFarm) : deux clics simultanés sur le même lien."""
import sys, threading, time
import psycopg

JETON = sys.argv[1]
depart = threading.Barrier(2)                    # les deux fils attendent d'être prêts
resultats = {}


def reinitialiser(nom):
    try:
        with psycopg.connect(dbname="greenfarm") as conn:
            depart.wait()                        # partez !
            row = conn.execute(
                "SELECT id, compte_id, utilise_le FROM jeton_reinit WHERE valeur = %s",
                (JETON,)).fetchone()
            if row is None or row[2] is not None:                # 1. vérifier
                resultats[nom] = "refusé : lien déjà utilisé"
                return
            time.sleep(0.2)
            conn.execute("UPDATE jeton_reinit SET utilise_le = now() WHERE id = %s", (row[0],))
            conn.execute("INSERT INTO changement_mdp (compte_id, jeton_id) VALUES (%s, %s)",
                         (row[1], row[0]))                       # 2. agir
            resultats[nom] = "accepté : mot de passe changé"
    except psycopg.Error as e:
        resultats[nom] = "erreur de la base : " + e.diag.message_primary


fils = [threading.Thread(target=reinitialiser, args=(n,)) for n in ("clic 1", "clic 2")]
for f in fils: f.start()
for f in fils: f.join()
for nom in sorted(resultats):
    print(nom, "->", resultats[nom])
with psycopg.connect(dbname="greenfarm") as conn:
    n = conn.execute("""SELECT count(*) FROM changement_mdp c
                        JOIN jeton_reinit j ON j.id = c.jeton_id
                        WHERE j.valeur = %s""", (JETON,)).fetchone()[0]
print("changements de mot de passe avec ce lien :", n)
```

La version corrigée remplace « vérifier, puis agir » par **une seule instruction**, qui vérifie et agit en même temps :

```
            row = conn.execute(
                """UPDATE jeton_reinit SET utilise_le = now()
                   WHERE valeur = %s AND utilise_le IS NULL AND expire_le > now()
                   RETURNING id, compte_id""", (JETON,)).fetchone()
            if row is None:
                resultats[nom] = "refusé : lien déjà utilisé ou expiré"
                return
```

Terminal — dossier-bdd

```
$ python course_reinit.py RESET-c.petit-10
clic 1 -> accepté : mot de passe changé
clic 2 -> accepté : mot de passe changé
changements de mot de passe avec ce lien : 2
$ python course_reinit_corrige.py RESET-c.petit-11
clic 1 -> refusé : lien déjà utilisé ou expiré
clic 2 -> accepté : mot de passe changé
changements de mot de passe avec ce lien : 1
```

Version vulnérable : 2 changements. Version corrigée : 1 seul

À vous · course_jeton.py et sa correction

1. **Dans VS Code**, créez `course_jeton.py` sur le modèle de l'exemple, avec les tables `jeton` et `session_app`. **Dans le terminal** (avec les variables `PGUSER`, `PGPASSWORD`, `PGHOST` d'hier) : `python course_jeton.py RT-f.ba-10`, puis `RT-f.ba-11`. **Attendu :** `sessions ouvertes avec ce jeton : 2`, à chaque fois. Si vous obtenez 1, passez `time.sleep` à 0.5.
2. Créez `course_jeton_corrige.py` avec l'`UPDATE … RETURNING`, et lancez-le sur `RT-f.ba-12` et `RT-f.ba-13`. **Attendu :** un `accepté`, un `refusé`, et 1 session, à chaque fois.

Étape 9.4 · Comprendre les quatre corrections (20 min)

Exemple GreenFarm · Verrouiller, ou tout faire en une instruction

`SELECT … FOR UPDATE` lit la ligne **et la verrouille**. Le terminal B attend ; quand A valide, B voit le lien déjà utilisé.

Correction 1 — SELECT … FOR UPDATE

	Terminal A (clic 1)	Terminal B (clic 2)
T1	`greenfarm=# BEGIN;` `BEGIN`	
T2		`greenfarm=# BEGIN;` `BEGIN`

T3	`greenfarm=*# SELECT valeur, utilise_le FROM jeton_reinit`
	`greenfarm*# WHERE valeur = 'RESET-c.petit-2' FOR UPDATE;`
	`valeur | utilise_le`
	`----------+-----------`
	`RESET-c.petit-2 |`
	`(1 row)`	

T4		`greenfarm=*# SELECT valeur, utilise_le FROM jeton_reinit`
		`greenfarm*# WHERE valeur = 'RESET-c.petit-2' FOR UPDATE;`
		`… en attente : la ligne est verrouillée par l'autre terminal`

T5	`greenfarm=*# UPDATE jeton_reinit SET utilise_le = now()`
	`greenfarm*# WHERE valeur = 'RESET-c.petit-2';`
	`UPDATE 1`	

T6	`greenfarm=*# COMMIT;` `COMMIT`	

T7		`… débloqué :`
		`valeur | utilise_le`
		`----------+----------`
		`RESET-c.petit-2 | 2026-11-09 09:52:21.831849+01`
		`(1 row)`

T8		`greenfarm=*# ROLLBACK;` `ROLLBACK`

Correction 1 : B attend, puis voit utilise_le rempli

L'`UPDATE … WHERE utilise_le IS NULL` verrouille lui aussi la ligne. Quand A valide, PostgreSQL **réévalue** la condition pour B : elle n'est plus vraie, 0 ligne.

Correction 2 — une seule instruction : UPDATE … RETURNING

	Terminal A (clic 1)	Terminal B (clic 2)
T1	`greenfarm=# BEGIN;` `BEGIN`	
T2		`greenfarm=# BEGIN;` `BEGIN`

T3	`greenfarm=*# UPDATE jeton_reinit SET utilise_le = now()`
	`greenfarm*# WHERE valeur = 'RESET-c.petit-3' AND utilise_le IS NULL`
	`greenfarm*#   AND expire_le > now()`
	`greenfarm*# RETURNING id;`
	`id`
	`---`
	`6a090706-6621-45df-8e2e-7eb1c100ed62`
	`(1 row)`	

T4		`greenfarm=*# UPDATE jeton_reinit SET utilise_le = now()`
		`greenfarm*# WHERE valeur = 'RESET-c.petit-3' AND utilise_le IS NULL`
		`greenfarm*#   AND expire_le > now()`
		`greenfarm*# RETURNING id;`
		… en attente : la ligne est verrouillée par l'autre terminal

T5	`greenfarm=*# COMMIT;` `COMMIT`	

T6		*… débloqué :*
		`id`
		`---`
		`(0 rows)`

T7		`greenfarm=*# COMMIT;` `COMMIT`

Correction 2 : A reçoit un id, B reçoit 0 ligne

Exemple GreenFarm · La base refuse le doublon, quel que soit le code

Une contrainte `UNIQUE (jeton_id)` interdit deux changements pour un même lien. Elle est d'abord refusée, car les tests ont laissé des doublons : on les nettoie, puis on la pose. Même le programme **vulnérable** ne peut plus réussir deux fois.

psql — greenfarm

```
greenfarm=# ALTER TABLE changement_mdp ADD CONSTRAINT changement_jeton_unique UNIQUE (jeton_id);
ERROR:  could not create unique index "changement_jeton_unique"
DETAIL:  Key (jeton_id)=(87d81c9b-88ef-46c5-a6f6-6b26b211551f) is duplicated.
greenfarm=# -- on garde le plus ancien changement de chaque lien
greenfarm-# DELETE FROM changement_mdp c
greenfarm-# USING changement_mdp d
greenfarm-# WHERE d.jeton_id = c.jeton_id AND d.change_le < c.change_le;
DELETE 2
greenfarm=# ALTER TABLE changement_mdp ADD CONSTRAINT changement_jeton_unique UNIQUE (jeton_id);
ALTER TABLE
```

La contrainte refusée, les doublons supprimés, la contrainte posée

Terminal — dossier-bdd

```
$ python course_reinit.py RESET-c.petit-12
clic 1 -> accepté : mot de passe changé
clic 2 -> erreur de la base : duplicate key value violates unique constraint "changement_jeton_unique"
changements de mot de passe avec ce lien : 1
```

Le code vulnérable, relancé : un seul changement

Exemple GreenFarm · Monter le niveau d'isolation

En `REPEATABLE READ`, la seconde modification de la même ligne échoue (`could not serialize access`) : l'application doit annuler et réessayer, et elle verra alors le lien utilisé.

Correction 4 — REPEATABLE READ

	Terminal A (clic 1)	Terminal B (clic 2)
T1	`greenfarm=# BEGIN ISOLATION LEVEL REPEATABLE READ;` `BEGIN`	
T2		`greenfarm=# BEGIN ISOLATION LEVEL REPEATABLE READ;` `BEGIN`

T3	`greenfarm=*# SELECT valeur, utilise_le FROM jeton_reinit`
	`greenfarm*# WHERE valeur = 'RESET-c.petit-4';`
	`valeur | utilise_le`
	`----------+----------`
	`RESET-c.petit-4 |`
	`(1 row)`	

T4		`greenfarm=*# SELECT valeur, utilise_le FROM jeton_reinit`
		`greenfarm*# WHERE valeur = 'RESET-c.petit-4';`
		`valeur | utilise_le`
		`----------+----------`
		`RESET-c.petit-4 |`
		`(1 row)`

T5	`greenfarm=*# UPDATE jeton_reinit SET utilise_le = now()`
	`greenfarm*# WHERE valeur = 'RESET-c.petit-4';`
	`UPDATE 1`	

T6		`greenfarm=*# UPDATE jeton_reinit SET utilise_le = now()`
		`greenfarm*# WHERE valeur = 'RESET-c.petit-4';`
		… en attente : la ligne est verrouillée par l'autre terminal

T7	`greenfarm=*# COMMIT;` `COMMIT`	
T8		*… débloqué :* `ERROR: could not serialize access due to concurrent update`
T9		`greenfarm=!# ROLLBACK;` `ROLLBACK`

Correction 4 : B échoue au lieu d'agir

À vous · Les quatre corrections

1. Rejouez les deux terminaux avec `RT-f.ba-2`, en ajoutant `FOR UPDATE` aux deux `SELECT`. **Attendu :** au T4, B se fige ; dès que A fait `COMMIT`, B voit `utilise_le` rempli. Faites `ROLLBACK` dans B.
2. Rejouez avec `RT-f.ba-3` et la seule instruction `UPDATE … RETURNING id`. **Attendu :** A reçoit un `id`, B attend puis reçoit 0 ligne.
3. Ajoutez `ALTER TABLE session_app ADD CONSTRAINT session_jeton_unique UNIQUE (jeton_id);`. **Attendu :** d'abord un refus (`could not create unique index`) ; supprimez les doublons en gardant la plus ancienne session de chaque jeton, recommencez, puis relancez `python course_jeton.py RT-f.ba-14`. **Attendu :** 1 session, et une `erreur de la base : duplicate key…` pour l'autre clic. Ajoutez la contrainte dans `jeton.sql`.
4. Rejouez avec `RT-f.ba-4`, en ouvrant par `BEGIN ISOLATION LEVEL REPEATABLE READ;`, sans `FOR UPDATE`. **Attendu :** l'`UPDATE` de B échoue avec `could not serialize access due to concurrent update`.
5. Dans `concurrence.md`, remplissez :

Correction	Où agit-elle ?	Que voit la seconde requête ?	Avantage	Limite
`SELECT … FOR UPDATE`	___	___	___	___
`UPDATE … WHERE … RETURNING`	___	___	___	___
`UNIQUE (jeton_id)`	___	___	___	___
`REPEATABLE READ`	___	___	___	___

Et répondez : le trigger du lab 6 est aussi un « vérifier, puis agir ». Deux insertions simultanées peuvent-elles le contourner ? Quelle correction le protégerait ?

Étape 9.5 · Durabilité et sauvegarde (15 min)

Exemple GreenFarm · Les réglages, puis la sauvegarde vérifiée

`fsync`, `synchronous_commit` à `on` : un `COMMIT` ne répond qu'une fois le journal d'écriture (WAL) enregistré sur le disque.

psql — greenfarm

```
greenfarm=# SHOW fsync;
 fsync
-------
 on
(1 row)
greenfarm=# SHOW synchronous_commit;
 synchronous_commit
--------------------
 on
(1 row)
greenfarm=# SHOW wal_level;
 wal_level
-----------
 replica
(1 row)
```

Les trois réglages de durabilité

Puis on sauvegarde, on prend l'**empreinte** du fichier (SHA-256), on restaure dans une autre base, on vérifie. Le journal restauré reste protégé, et l'empreinte prouve que la sauvegarde n'a pas été modifiée.

Terminal — dossier-bdd

```
$ pg_dump -U postgres -d greenfarm -Fc -f greenfarm_j2.dump

$ sha256sum greenfarm_j2.dump > greenfarm_j2.dump.sha256

$ cat greenfarm_j2.dump.sha256
a52973b5d401a33f1d8058e85765d06cf23dd21a9127229a525ba84c8dde1ce2  greenfarm_j2.dump
$ createdb -U postgres greenfarm_restore

$ pg_restore -U postgres -d greenfarm_restore greenfarm_j2.dump

$ psql -U postgres -d greenfarm_restore -c "SELECT count(*) AS journal FROM journal_audit"
 journal
---------
       2
(1 row)
$ psql -U postgres -d greenfarm_restore -c "DELETE FROM journal_audit"
ERROR:  journal_audit est en ajout seul : DELETE interdit
CONTEXT:  PL/pgSQL function journal_interdit() line 3 at RAISE
$ sha256sum -c greenfarm_j2.dump.sha256
greenfarm_j2.dump: OK
```

Sauvegarde, empreinte, restauration : le journal est intact et toujours protégé, l'empreinte est OK

À vous · durabilite.md

1. Dans psql : `SHOW fsync;`, `SHOW synchronous_commit;`, `SHOW wal_level;`. **Attendu :** `on`, `on`, `replica`.
2. **Dans le terminal**, refaites la sauvegarde de l'exemple pour `queenbee` (`queenbee_j2.dump`, restauration dans `queenbee_restore`). Sous Windows, remplacez `sha256sum` par `certutil -hashfile queenbee_j2.dump SHA256` et comparez à l'œil. **Attendu :** le même nombre de lignes de journal, le `DELETE` refusé dans la copie, et `queenbee_j2.dump: OK`.
3. **Dans VS Code**, créez `durabilite.md` : ce qu'on perdrait avec `synchronous_commit = off` ; pourquoi on garde l'empreinte **ailleurs** que la sauvegarde ; ce que `pg_dump` ne sauvegarde pas (les rôles : `pg_dumpall --roles-only`) ; la différence entre durabilité (survivre à une panne) et sauvegarde (survivre à la perte du disque, à une erreur, à un rançongiciel).

Ne commitez **pas** `queenbee_j2.dump` : il est gros et contient des données personnelles. Commitez seulement son empreinte.

Enregistrez le lab :

```
git add jeton.sql course_jeton.py course_jeton_corrige.py concurrence.md durabilite.md
queenbee_j2.dump.sha256
git commit -m "Lab 9 : transactions, course au jeton et sauvegarde vérifiée"
```

Testez-vous · Lab 9

1. Pourquoi l'invite devient-elle `queenbee=!#` après une erreur dans une transaction ? Que faut-il taper ?
2. Pourquoi `READ COMMITTED` ne protège-t-il pas de la course au jeton ?
3. Pourquoi `UPDATE … WHERE utilise_le IS NULL RETURNING id` est-il sûr, sans verrou explicite ?
4. Un rançongiciel chiffre le serveur. ACID vous protège-t-il ? Qu'est-ce qui vous protège ?

10. Lab 10 · Revue de sécurité (1 h)

Pourquoi ce lab. Hier, la requête paramétrée a arrêté l'injection. Aujourd'hui, vous allez voir deux failles qu'elle **n'arrête pas**. Puis vous relisez toute votre base avec une grille de contrôle, comme un auditeur, et vous corrigez ce que vous trouvez.

Étape 10.1 · L'injection par le tri (20 min)

> **Un %s protège une valeur, pas un nom de colonne**
>
> Le `%s` de psycopg remplace une **valeur** (un code, une date). Il ne peut pas remplacer un nom de colonne ou un sens de tri. Le développeur pressé recopie alors la saisie : `"ORDER BY " + tri`. L'attaquant n'a même plus besoin d'apostrophe. La correction : une **liste blanche** de colonnes connues, `sql.Identifier`, et jamais le message d'erreur de la base affiché à l'utilisateur.

Exemple GreenFarm · Trois attaques par le tri

`recherche_serres_vuln.py` demande une exploitation (paramétrée) et un tri (recopié dans la requête) :

```
sql = ("SELECT code, exploitation_id, surface_m2 FROM serre "
       "WHERE exploitation_id = %s ORDER BY " + tri)        # valeur paramétrée, tri recopié
try:
    for row in conn.execute(sql, (exploitation,)):
        print(row)
except psycopg.Error as e:
    print("Erreur :", e)                                    # affiche l'erreur de la base
```

Par erreur : un `cast(… AS int)` sur le téléphone. La conversion échoue, et le message d'erreur **contient** le téléphone.

```
Terminal — dossier-bdd

$ python recherche_serres_vuln.py
Exploitation : FR-BEAU
Trier par (code ou surface_m2) : code
('FR-BEAU-01', 'FR-BEAU', 400)
('FR-BEAU-02', 'FR-BEAU', 250)
$ python recherche_serres_vuln.py
Exploitation : FR-BEAU
Trier par (code ou surface_m2) : cast((SELECT telephone FROM exploitant WHERE nom = 'Claire Petit') AS int)
Erreur : invalid input syntax for type integer: "06 22 33 44 55"
```

Une recherche normale, puis le téléphone de Claire Petit dans le message d'erreur

À l'aveugle : un `CASE WHEN (question) THEN code ELSE surface_m2::text END`. L'**ordre** des lignes répond oui ou non. Ici : les chiffres 4 et 5 du téléphone sont-ils `22` ? Oui (tri par code). `23` ? Non (tri par surface). Chiffre par chiffre, l'attaquant reconstitue tout le numéro.

```
Terminal — dossier-bdd

$ python recherche_serres_vuln.py
Exploitation : FR-BEAU
Trier par (code ou surface_m2) : CASE WHEN (SELECT substr(telephone, 4, 2) FROM exploitant WHERE nom = 'Claire Peti
t') = '22' THEN code ELSE surface_m2::text END
('FR-BEAU-01', 'FR-BEAU', 400)
('FR-BEAU-02', 'FR-BEAU', 250)
$ python recherche_serres_vuln.py
Exploitation : FR-BEAU
Trier par (code ou surface_m2) : CASE WHEN (SELECT substr(telephone, 4, 2) FROM exploitant WHERE nom = 'Claire Peti
t') = '23' THEN code ELSE surface_m2::text END
('FR-BEAU-02', 'FR-BEAU', 250)
('FR-BEAU-01', 'FR-BEAU', 400)
```

L'ordre des lignes change selon la réponse

Empilée : `code; DROP TABLE journal_audit`. Elle échoue : une requête qui a des paramètres ne peut contenir qu'une commande. La version corrigée, `recherche_serres.py`, choisit le tri dans une liste blanche :

```
TRIS = {"code": sql.Identifier("code"), "surface_m2": sql.Identifier("surface_m2")}
if tri not in TRIS:
    print("Tri refusé : choisissez code ou surface_m2")
    raise SystemExit(1)
requete = sql.SQL("SELECT code, exploitation_id, surface_m2 FROM serre "
                  "WHERE exploitation_id = %s ORDER BY {}").format(TRIS[tri])
```

```
Terminal — dossier-bdd

$ python recherche_serres_vuln.py
Exploitation : FR-BEAU
Trier par (code ou surface_m2) : code; DROP TABLE journal_audit
Erreur : cannot insert multiple commands into a prepared statement
$ python recherche_serres.py
Exploitation : FR-BEAU
Trier par (code ou surface_m2) : code
('FR-BEAU-01', 'FR-BEAU', 400)
('FR-BEAU-02', 'FR-BEAU', 250)
$ python recherche_serres.py
Exploitation : FR-BEAU
Trier par (code ou surface_m2) : cast((SELECT telephone FROM exploitant WHERE nom = 'Claire Petit') AS int)
Tri refusé : choisissez code ou surface_m2
```

L'attaque empilée échoue ; la version corrigée refuse le tri inconnu

À vous · recherche_vuln.py, puis recherche.py

1. **Dans VS Code**, créez `recherche_vuln.py` : il demande un site (paramétré) et un tri (`code` ou `date_installation`, recopié). Testez `SN-CASA` puis `code`. **Attendu :** les quatre ruches de SN-CASA.
2. Attaquez-le de trois façons, et collez chaque saisie et chaque réponse dans `rapport_revue.md` :
   - **par erreur** : le téléphone de Fatou Ba. **Attendu :** `invalid input syntax for type integer: "+221 77 100 00 02"` ;
   - **à l'aveugle** : `CASE WHEN (SELECT substr(telephone, 6, 2) FROM apicultrice WHERE nom = 'Fatou Ba') = '77' THEN code ELSE longitude::text END`, puis avec `'78'`. **Attendu :** l'ordre change ;
   - **empilée** : `code; DROP TABLE journal_audit`. **Attendu :** `cannot insert multiple commands into a prepared statement`.
3. Créez `recherche.py`, avec une liste blanche et `sql.Identifier`, sans afficher l'erreur de la base. Rejouez l'attaque par erreur. **Attendu :** `Tri refusé : choisissez code ou date_installation`.

Étape 10.2 · La vue qui en dit trop (15 min)

Exemple GreenFarm · La vue trop large

Un collègue ajoute une vue « pour la carte des serres », avec le nom et le téléphone de **tous** les exploitants, et la donne au groupe `gf_exploitant`. `c.petit` n'a aucun droit sur `exploitant`… mais la vue s'exécute avec les droits de son propriétaire : elle lit tout. C'est une **élévation de privilège**.

psql — greenfarm

```
greenfarm=# SET ROLE "c.petit";
SET
greenfarm=> SELECT nom, telephone FROM exploitant;
ERROR:  permission denied for table exploitant
greenfarm=> SELECT code, nom, telephone FROM v_carte_serres;
    code    |      nom      |   telephone
------------+---------------+---------------
 FR-BEAU-01 | Paul Martin   | 06 11 22 33 44
 FR-BEAU-02 | Claire Petit  | 06 22 33 44 55
 FR-PROV-01 | Paul Martin   | 06 99 88 77 66
 FR-LOIR-01 | Pierre Martin | 06 55 44 33 22
 FR-LOIR-02 | Inès Roux     | 06 33 44 55 66
(5 rows)
greenfarm=> RESET ROLE;
RESET
```

La table est refusée, la vue montre tous les téléphones

Exemple GreenFarm · La fonction espion

Même une vue bien filtrée peut fuir. Pour aller vite, PostgreSQL peut évaluer une condition de l'utilisateur **avant** le filtre de la vue. Si cette condition est une fonction qui affiche ce qu'elle reçoit, elle voit **toutes** les lignes.

psql — greenfarm

```
greenfarm=# SET ROLE "c.petit";
SET
greenfarm=> SELECT code FROM v_mes_serres;
    code
------------
 FR-BEAU-02
(1 row)
greenfarm=> CREATE FUNCTION pg_temp.espion(c text, s int) RETURNS boolean
greenfarm-> LANGUAGE plpgsql COST 0.0000001 AS $$
greenfarm-> BEGIN
greenfarm->   RAISE NOTICE 'vu : % (% m2)', c, s;
greenfarm->   RETURN true;
greenfarm-> END $$;
CREATE FUNCTION
greenfarm=> SELECT code FROM v_mes_serres WHERE pg_temp.espion(code, surface_m2);
NOTICE:  vu : FR-BEAU-01 (400 m2)
NOTICE:  vu : FR-BEAU-02 (250 m2)
NOTICE:  vu : FR-PROV-01 (500 m2)
NOTICE:  vu : FR-LOIR-01 (300 m2)
NOTICE:  vu : FR-LOIR-02 (300 m2)
    code
------------
 FR-BEAU-02
(1 row)
```

Le résultat ne montre qu'une serre, mais les NOTICE affichent les cinq

Les corrections, dans un second terminal en `postgres` : la vue de carte réécrite (ses serres seulement, sans téléphone), `security_barrier` sur toutes les vues filtrantes (le filtre de la vue passe d'abord), et `REVOKE TEMPORARY ON DATABASE greenfarm FROM PUBLIC` (plus de fonction dans `pg_temp`).

```
DROP VIEW v_carte_serres;
CREATE VIEW v_carte_serres WITH (security_barrier = true) AS
SELECT s.code, s.surface_m2
FROM serre s JOIN compte c ON c.exploitant_id = s.exploitant_id
WHERE c.login = qui();
GRANT SELECT ON v_carte_serres TO gf_exploitant;
ALTER VIEW v_moi               SET (security_barrier = true);
ALTER VIEW v_mes_serres        SET (security_barrier = true);
ALTER VIEW v_mes_interventions SET (security_barrier = true);
REVOKE TEMPORARY ON DATABASE greenfarm FROM PUBLIC;
```

psql — greenfarm

```
greenfarm=> SELECT * FROM v_carte_serres;
    code    | surface_m2
------------+------------
 FR-BEAU-02 |        250
(1 row)
greenfarm=> SELECT code FROM v_mes_serres WHERE pg_temp.espion(code, surface_m2);
NOTICE:  vu : FR-BEAU-02 (250 m2)
    code
------------
 FR-BEAU-02
(1 row)
greenfarm=> RESET ROLE;
RESET
```

Retest dans la même session : une seule serre, et l'espion ne voit plus qu'elle

psql — greenfarm

```
greenfarm=# SET ROLE "c.petit";
SET
greenfarm=> CREATE FUNCTION pg_temp.espion(c text, s int) RETURNS boolean
greenfarm-> LANGUAGE plpgsql COST 0.0000001 AS $$
greenfarm-> BEGIN
greenfarm->   RAISE NOTICE 'vu : % (% m2)', c, s;
greenfarm->   RETURN true;
greenfarm-> END $$;
ERROR:  permission denied to create temporary tables in database "greenfarm"
greenfarm=> RESET ROLE;
RESET
```

Dans une nouvelle session, plus de fonction temporaire possible

À vous · La vue de carte, l'espion, les corrections

1. Ajoutez la vue du collègue pressé : `v_carte_ruches` (code, latitude, longitude, nom et téléphone de **toutes** les ruches), donnée à `qb_apicultrice`. En `f.ba`, lisez `apicultrice`, puis la vue. **Attendu :** la table est refusée, la vue montre les 11 ruches avec tous les téléphones.
2. Toujours en `f.ba`, jouez l'espion sur `v_mes_ruches` avec `pg_temp.espion(code, latitude, longitude)`. **Attendu :** le résultat ne montre que `SN-CASA-013`, mais les `NOTICE` donnent la **position GPS des 11 ruches**. Pour un voleur, c'est la carte complète. Gardez cette session ouverte.
3. **Dans VS Code**, créez `corrections.sql` (la vue réécrite, `security_barrier` sur toutes vos vues filtrantes, `REVOKE TEMPORARY`), et lancez-le dans un **second** terminal, en `postgres`.
4. Retestez dans la session ouverte, puis dans une nouvelle session `f.ba`. **Attendu :** une seule ligne sans téléphone ; l'espion ne voit plus que `SN-CASA-013` ; `permission denied to create temporary tables`.
5. Dans `rapport_revue.md`, section « Notre propre code », un constat par faille : preuve, correction, retest.

Étape 10.3 · Auditer votre propre base (25 min)

> **Écrire un constat**
>
> Un bon constat a : un **identifiant** (R1, R2…) ; une **gravité** (critique : fuite de données personnelles ou de preuves ; haute : une règle métier contournée ; moyenne : une défense en profondeur absente ; basse : une bonne pratique) ; une **preuve** (la commande exacte et son résultat, que n'importe qui peut rejouer) ; une **correction** ; un **retest**. Un constat sans preuve n'est pas un constat.

Exemple GreenFarm · Les trois premiers contrôles de la grille

psql — greenfarm

```
greenfarm=# -- C1 : fonctions exécutables par PUBLIC
greenfarm-# SELECT proname FROM pg_proc
greenfarm-# WHERE pronamespace = 'public'::regnamespace
greenfarm-#   AND has_function_privilege('public', oid, 'EXECUTE');
 proname
---------
(0 rows)
greenfarm=# -- C2 : les SECURITY DEFINER fixent leur search_path
greenfarm-# SELECT proname, proconfig FROM pg_proc
greenfarm-# WHERE pronamespace = 'public'::regnamespace AND prosecdef;
   proname   |            proconfig
-------------+---------------------------------
 audit_ligne | {"search_path=public, pg_temp"}
(1 row)
greenfarm=# -- C3 : les vues filtrantes sont des barrières
greenfarm-# SELECT relname, reloptions FROM pg_class
greenfarm-# WHERE relkind = 'v' AND relnamespace = 'public'::regnamespace
greenfarm-# ORDER BY relname;
       relname       |                  reloptions
---------------------+----------------------------------------------
 v_carte_serres      | {security_barrier=true}
 v_export_recherche  |
 v_mes_interventions | {check_option=cascaded,security_barrier=true}
 v_mes_serres        | {security_barrier=true}
 v_moi               | {security_barrier=true}
(5 rows)
```

C1 : aucune fonction ouverte à PUBLIC ; C2 : search_path fixé ; C3 : les vues filtrantes sont des barrières

`v_export_recherche` n'a pas `security_barrier` : ce n'est pas une faute, car elle ne filtre pas par utilisateur et n'est donnée à personne. Un bon auditeur explique, il ne coche pas seulement.

À vous · Passer la grille sur votre base

1. **Passez la grille** sur votre base `queenbee`. Pour chaque ligne, lancez la commande, notez le résultat, et ouvrez un constat si le contrôle échoue.

N°	Contrôle	Comment vérifier	Attendu
C1	aucune fonction exécutable par `PUBLIC`	requête C1 de l'exemple	0 ligne
C2	toute fonction `SECURITY DEFINER` fixe son `search_path`	requête C2 de l'exemple	`search_path=…` partout
C3	les vues filtrantes sont des barrières	requête C3 de l'exemple	`security_barrier=true`
C4	aucune vue ne montre des données hors périmètre	lire chaque vue en `SET ROLE` apicultrice	ses lignes seulement
C5	le journal est en ajout seul	`UPDATE`, `DELETE`, `TRUNCATE`	trois refus

N°	Contrôle	Comment vérifier	Attendu
C6	le journal nomme le vrai acteur	une action en `SET ROLE`, puis le journal	le bon login
C7	une apicultrice n'écrit pas chez une autre	test 4 du lab 8	refus
C8	un jeton ne sert qu'une fois	`course_jeton.py` sur un jeton neuf	1 session
C9	pas de requête construite par `+` ou `f"`	lire vos scripts Python	aucune
C10	les preuves ne partent pas en cascade	requête `confdeltype` du lab 8	`RESTRICT`
C11	pas de mot de passe dans le dépôt	`git grep -i password`	à discuter
C12	la base se reconstruit avec vos fichiers	les relancer dans une base vide	aucune erreur

2. **Rédigez le rapport** dans `rapport_revue.md` :

```
# Revue de sécurité de notre base — Groupe ___

## Synthèse
Contrôles passés : ___ / 12   Constats : ___ critique(s), ___ haute(s), ___ moyenne(s), ___
basse(s)

## Failles démontrées (étapes 10.1 et 10.2)
| ID | Faille | Preuve | Correction | Retest |
|----|--------|--------|------------|--------|
| P1 | injection par le tri | ___ | ___ | ___ |
| P2 | vue v_carte_ruches trop large | ___ | ___ | ___ |
| P3 | fuite par fonction espion | ___ | ___ | ___ |

## Constats de la grille
| ID | Contrôle | Gravité | Constat | Preuve (commande et résultat) | Correction | Retest |
|----|----------|---------|---------|-------------------------------|------------|--------|
| R1 | ___      | ___     | ___     | ___                           | ___        | ___    |
```

3. **Corrigez.** Ajoutez chaque correction à `corrections.sql`, appliquez-la, puis rejouez la preuve et notez le retest.

Enregistrez le lab :

```
git add rapport_revue.md recherche_vuln.py recherche.py corrections.sql
git commit -m "Lab 10 : revue de sécurité, injection par le tri, vues corrigées"
```

Testez-vous · Lab 10

1. Pourquoi `conn.execute("… ORDER BY %s", (tri,))` ne trie-t-il pas par la colonne choisie ?
2. L'injection empilée a échoué : `recherche_vuln.py` est-il sûr pour autant ?
3. Quelle différence entre `security_barrier` et `security_invoker` ?
4. Classez par gravité : un `README` absent ; la vue `v_carte_ruches` d'origine ; une fonction `SECURITY DEFINER` sans `search_path`.

11. Rendu du dossier de conception (30 min)

Dernière vérification, comme hier : on doit pouvoir reconstruire votre base avec vos seuls fichiers. Tout se fait **dans le terminal**, depuis `dossier-bdd`.

Étape 11.1 · Exporter le schéma complet (tables, vues, fonctions, triggers, droits) :

```
mkdir -p jour2/1_schema_normalise jour2/2_roles_privileges jour2/3_trigger_audit
jour2/4_test_concurrence jour2/5_revue_securite
pg_dump -U postgres -d queenbee --schema-only -f jour2/1_schema_normalise/schema_j2.sql
```

Ouvrez `schema_j2.sql` dans VS Code : vous devez y trouver `CREATE TRIGGER recolte_audit`, `security_barrier` et `GRANT SELECT ON TABLE public.v_mes_ruches`.

Étape 11.2 · Créer `tests_audit.sql` avec les tests de l'étape 8.3 (points 2 à 4) et leurs résultats en commentaire.

Étape 11.3 · Ranger les fichiers :

```
git mv formation.sql volume.sql denormalisation.sql references.sql check.sql normalisation_j2.md
jour2/1_schema_normalise/
git mv roles.sql matrice_droits.md tests_contournement.sql jour2/2_roles_privileges/
git mv audit.sql jour2/3_trigger_audit/
mv tests_audit.sql jour2/3_trigger_audit/
git mv jeton.sql course_jeton.py course_jeton_corrige.py concurrence.md durabilite.md
queenbee_j2.dump.sha256 jour2/4_test_concurrence/
git mv rapport_revue.md recherche_vuln.py recherche.py corrections.sql jour2/5_revue_securite/
```

Si `git mv` répond `not under version control`, le fichier n'a jamais été ajouté : `git add`, puis recommencez.

Étape 11.4 · Écrire `jour2/README.md` dans VS Code :

```
# Queen Bee — Dossier de conception, jour 2 — Groupe ___

## Les 5 parties
1. Schéma normalisé : 1_schema_normalise/
2. Rôles et privilèges : 2_roles_privileges/ (4 rôles, ___ vues, 1 fonction de validation)
3. Trigger d'audit : 3_trigger_audit/ (tables auditées : ___)
4. Test de concurrence : 4_test_concurrence/ (2 sessions avant, 1 après)
5. Revue de sécurité : 5_revue_securite/ (___ constats, ___ corrigés)

## Reconstruire (après la base du jour 1, avec -v ON_ERROR_STOP=1)
1. formation.sql   2. volume.sql (une fois)   3. denormalisation.sql
4. references.sql puis check.sql   5. audit.sql   6. roles.sql
7. jeton.sql   8. corrections.sql

## Nos choix principaux
- ___ (ex. : BCNF + trigger pour les formations, parce que ___)
- ___ (ex. : la synthèse est rafraîchie toutes les ___ ; le vol est détecté sur ___)

## Limites connues
- un superutilisateur peut désactiver les triggers du journal : ___
```

Étape 11.5 · Tester la reconstruction dans une base vide, en suivant votre `README` : la base du jour 1, puis les huit fichiers du jour 2. **Attendu :** aucune `ERROR`, puis les tests de contournement donnent les mêmes résultats que sur votre base de travail.

Étape 11.6 · Archiver et rendre :

```
git add -A
git commit -m "Jour 2 : dossier de conception"
git archive --format=zip -o GROUPE_X_jour2.zip HEAD jour2
```

Contrôle	Fait ?
le découpage BCNF est prouvé sans perte (0 ligne), et la règle perdue est rétablie	☐
la revue BCNF de toutes les tables est dans `normalisation_j2.md`	☐
les deux temps `EXPLAIN ANALYZE` et le retard de la synthèse sont notés	☐
l'export du bailleur ne contient ni nom, ni téléphone, ni position précise	☐
la matrice des droits correspond aux `GRANT` de `roles.sql`	☐
les douze tests de contournement ont leur résultat en commentaire	☐
le journal nomme le vrai acteur et refuse `UPDATE`, `DELETE`, `TRUNCATE`	☐
la course au jeton est démontrée (2 sessions), puis corrigée (1 session)	☐
la sauvegarde est restaurée et son empreinte vérifiée	☐
le rapport de revue a des preuves rejouables, des corrections et des retests	☐
la base se reconstruit avec le `README`	☐